

Performance and Security in Userspace NVM File Systems through State Separation

DIYU ZHOU, Peking University, Beijing, China

VOJTECH ASCHENBRENNER, University of Wisconsin-Madison, Madison, United States

TAO LYU, EPFL, Lausanne, Switzerland

JIAN ZHANG, Hong Kong University of Science and Technology, Guangzhou, Guangzhou, China

SUDARSUN KANNAN, Rutgers University, Piscataway, United States

SANIDHYA KASHYAP, EPFL, Lausanne, Switzerland

Userspace library file systems (LibFSes) promise to unleash the performance potential of non-volatile memory (NVM) by directly accessing it and enabling unprivileged applications to customize their LibFSes. Unfortunately, such benefits pose a significant challenge to ensuring metadata integrity. Existing works either underutilize NVM's performance or forgo security guarantees.

We present TRIO, a userspace file system architecture that resolves this inherent tension. Our key insight is that other state (*i.e.*, auxiliary state) in a file system can be regenerated from its “ground truth” state (*i.e.*, core state). TRIO explicitly defines the data structure of a single core state and shares it as common knowledge among its LibFSes and the trusted entity. A LibFS directly accesses NVM without involving the trusted entity and can be customized with its private auxiliary state. The trusted entity enforces metadata integrity by verifying the core state of a file when its write access is transferred from one LibFS to another. We design a generic POSIX-like file system called ARCKFS and two customized file systems based on TRIO. Our evaluation shows that ARCKFS outperforms existing NVM file systems by 3.1× to 17× on LevelDB while the customized file systems further outperform ARCKFS by 1.3×.

CCS Concepts: • **Hardware** → **Non-volatile memory**; • **Information systems** → **Phase change memory**; • **Software and its engineering** → **File systems management**.

Additional Key Words and Phrases: Userspace File Systems, Library File Systems, Direct Access, File System Customization, File System Integrity, Persistent Memory

1 Introduction

Emerging non-volatile memory (NVM) technologies, *e.g.*, Intel Optane persistent memory [12] and future CXL-based storage devices [17, 43], offer the best of memory and storage. NVM's unique characteristics provide new opportunities to design high-performance file systems. Specifically, NVM allows direct access with unprivileged load and store instructions. Furthermore, the hardware memory management unit (MMU) enforces access permission to different NVM regions, thus deprecating the need for a privileged entity to mediate every NVM

¹This paper, originally titled “*Enabling High-Performance and Secure Userspace NVM File Systems with the Trio Architecture*”, was invited by ACM TOCS for extended publication through recommendation by the Chairs of SOSP 2023.

Authors' Contact Information: Diyu Zhou, Peking University, Beijing, China; e-mail: diyu.zhou@pku.edu.cn; VOJTECH ASCHENBRENNER, University of Wisconsin-Madison, Madison, Wisconsin, United States; e-mail: asch@cs.wisc.edu; Tao Lyu, EPFL, Lausanne, VD, Switzerland; e-mail: tao.lyu@epfl.ch; Jian Zhang, Hong Kong University of Science and Technology, Guangzhou, Guangdong, China; e-mail: jian@sosp.dev; Sudarsun Kannan, Rutgers University, Piscataway, United States; e-mail: sudarsun.kannan@rutgers.edu; Sanidhya Kashyap, EPFL, Lausanne, VD, Switzerland; e-mail: sanidhya.kashyap@epfl.ch.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 1557-7333/2026/7-ART

<https://doi.org/10.1145/3828543>

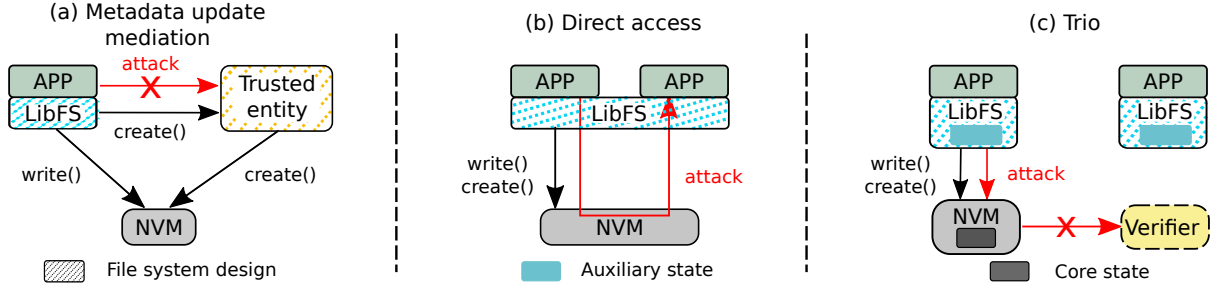


Fig. 1. Comparison of userspace NVM file system architectures. (a) Designs [33, 48, 53, 68, 70] that mediate metadata updates prevent direct NVM access and hinder unprivileged private customization. (b) Designs [38, 56] that support direct NVM access incur security vulnerabilities since malicious applications can corrupt shared file system state. (c) Trio enables direct NVM access, unprivileged private customization, and metadata integrity through state separation and a clean decouple of responsibilities.

access. These observations lead to userspace NVM file system designs that move (most) parts of the file system functionality out of the kernel to an application-linked library file system (LibFS) [33, 38, 48, 53, 56, 68].

Userspace NVM file systems bring two key performance advantages. First, an application can *directly access* NVM through the associated LibFS to perform file system operations, thereby minimizing the software overhead in the storage stack. Second, an application can *customize* its LibFS to bridge the semantic gaps and further boost its performance [68]. Critically, an application does not require special privileges to customize its LibFS. Moreover, customizing for one workload may often negatively affect another. Userspace NVM file systems can effectively avoid this issue by assigning private LibFSes to each application.

Unfortunately, the performance advantages also pose an inherent challenge to enforcing file system *security*. Specifically, with *direct access*, malicious applications can bypass enforcement and attack others by corrupting file system metadata. Resolving this requires a trusted entity to validate metadata integrity. However, due to *customization*, each LibFS' data structures may be different, which the trusted entity does not understand, thus preventing the validation.

Existing designs fail to resolve such inherent tension. The conventional design [33, 48, 53, 68, 70] employs a trusted entity to mediate and perform metadata updates. While this design ensures metadata integrity, it incurs expensive overhead for metadata updates (up to 68%, as reported in [38]), bakes file system design into the trusted entity, and therefore requires special privileges for customization. It thus significantly compromises the aforementioned performance advantages. On the other hand, ZoFS [38] enables direct metadata updates but forgoes metadata integrity. ZoFS offers protection by confining metadata corruption within one set of NVM pages (*i.e.*, coffer). Despite this, since applications share coffers, malicious applications can still attack by corrupting the metadata *within* a coffer (§2.3.2).

We present TRIO, a new userspace NVM file system architecture that provides both performance and security by supporting three crucial properties simultaneously: (1) **Direct access**: LibFSes can directly access NVM for both data and metadata operations on both regular files and directories. (2) **Unprivileged private customization**: Applications can flexibly customize their LibFSes without special privileges (thus being unprivileged) or affecting others (thus being private). (3) **Metadata integrity**: As with kernel file systems, a malicious application cannot attack others by corrupting the metadata state.

To overcome the inherent tension among the design goals, our key insight is that file system state can be *separated* into core state and auxiliary state. Core state is the essential state that a file system uses as ground truth (*e.g.*, inode, data pages), while auxiliary state is decided by and can be rebuilt from core state (*e.g.*, in-memory

cache, block bitmap). Using this insight, TRIO explicitly defines the layout and data structure of a single core state, which are shared among components while each component maintains its private auxiliary state.

TRIO consists of three (types of) components: an in-kernel access controller, per-application LibFSes, and a trusted userspace integrity verifier. The kernel controller decides which shared file system resources (*i.e.*, NVM pages and inodes) a LibFS can access. Each LibFS realizes a complete file system design, *directly accesses* a file²'s core state for data and metadata operations, and performs *unprivileged private customization* with its private auxiliary state.

However, with *direct access*, a malicious LibFS (or application) can corrupt the shared core state and thus attack others. This can be achieved by, *e.g.*, removing a non-empty directory to cause disconnected files, or making two files share the same data page, thereby causing data corruption.

Therefore, to ensure *secure sharing*, TRIO prevents LibFSes from updating a file simultaneously. Furthermore, before a LibFS accesses a file modified by another LibFS, the integrity verifier checks if the file's core state is valid. If not, the kernel controller handles the corruption. Unlike a full offline file system check, the integrity verifier only checks the core state of a single file, making its overhead acceptable (ranges from several to hundreds of microseconds for medium-sized files, as reported in §6.5). The LibFS then rebuilds the file's auxiliary state from the valid core state and can securely access the file. TRIO thus departs from the prior approaches that either validate metadata integrity for every operation [33, 48, 53, 68] or completely forgo it [38, 56].

We demonstrate the advantages of TRIO by designing a generic POSIX-like (but not fully POSIX-compliant) file system: ARCKFS³. To fully exploit the benefits of *direct access*, we carefully design ARCKFS with efficient data structures, scalable NVM data access engine, and fine-grained parallelism, thereby achieving low latency, high bandwidth, and excellent scalability for both data and metadata operations.

Using TRIO, we further design two customized file systems based on ARCKFS's core state: KVFS [68], which optimizes for small file access, and FPFS [67, 76], which optimizes for deep directory access. The customization involves changing interfaces and key metadata structures and is heavily workload-specific. By making file system design solely the responsibility of LibFS, TRIO enables the customization without special privilege or affecting other applications, neither of which prior approaches [68] could achieve. Our evaluation shows that ARCKFS outperforms other file systems by 3.1× to 17× on LevelDB, and has two orders of magnitude improvements in scalability microbenchmarks. KVFS and FPFS further outperform ARCKFS by 1.3×.

Limitations (§3.3). TRIO inherits two general limitations from prior userspace NVM file systems [33, 48, 53, 68]. First, file systems based on the TRIO architecture cannot fully conform to POSIX semantics. Second, TRIO incurs extra costs upon file sharing, which may not be suitable for workloads where multiple untrusted applications perform frequent concurrent writes to shared files.

This paper makes the following contributions:

- **TRIO.** We propose a new userspace file system architecture: TRIO. Through file system state separation and clean responsibility decoupling, TRIO provides *direct access*, *unprivileged private customization*, and *metadata integrity* simultaneously.
- **ARCKFS.** We design ARCKFS, a POSIX-like userspace NVM file system based on TRIO. ARCKFS achieves low latency, high throughput, and excellent scalability for both data and metadata operations.
- **File system customization.** We present two customized file systems, demonstrating the flexible unprivileged private customization enabled by TRIO.

²We use the term “file” to refer to both regular files and directories.

³named after Beethoven's Piano Trio “Archduke”

- D. Zhou et al.

	In-kernel	Metadata update mediation (§2.3.1)	Direct access (§2.3.2)	TRIO
Direct data access	×	✓	✓	✓
Direct metadata access	×	×	✓	✓
Unprivileged custom.	×	×	✓	✓
Per-app custom.	×	×	×	✓
Metadata integrity	✓	✓	×	✓

Table 1. Summary of NVM file systems. Prior works either mediate metadata updates and impede customization or do not enforce metadata integrity and thus reduce security. TRIO overcomes this tension and provides both performance and security.

2 Background and Motivation

Our work and many related NVM file systems are not limited to a specific NVM technology. This section presents our hardware assumptions on NVM (§2.1), a motivation for file system customization (§2.2), existing userspace NVM file systems designs (§2.3), and a summary of lessons we learned from prior designs (§2.4), which motivates TRIO.

2.1 NVM Technologies

Hardware assumptions. Across the paper, we use the term “NVM” to denote storage devices with the following characteristics. First, software can access NVM through unprivileged instructions (*e.g.*, load/store). Second, there exist mechanisms (*e.g.*, page tables) for privileged software to enforce access permission to parts of NVM. Third, the access latency of NVM is low and at the scale of at most hundreds of nanoseconds. Fourth, NVM is byte addressable. The first characteristic enables userspace access, while the second one ensures security. The third characteristic motivates minimizing software overhead, and the last characteristic is critical to our file system design.

Present and future use of NVM. Many storage devices meet the above assumptions. Specifically, industry has been using battery-backed DRAM as NVM [11, 50]. Intel Optane Persistent Memory [12], based on 3D Xpoint [1], is the first publicly available NVM technique. Emerging Compute Express Link (CXL) standard [4] introduces new possibilities to NVM. The current CXL standard includes specific support for NVM [3]. Industry has already proposed CXL-based NVM devices with new memory technologies [14, 43] or a combination of battery-based DRAM and flash memory [17].

2.2 File System Customization

It is widely known that a general-purpose system that provides acceptable performance under all scenarios cannot provide optimal performance for specific workloads. This observation has led to much work that improves application performance with workload-specific customization of the underlying system software [40, 46, 49, 61, 62, 68, 74]. Customization improves application performance by bridging the semantic gaps between the applications and the underlying system software.

Prior work has enabled workload-specific customization for thread and networking packet scheduling [49], kernel locks [61], or even the entire operating system [40, 46]. In addition, prior work has revealed two ideal characteristics for customization. First, customization should be performed securely without special privileges, and thus, all applications in the system can benefit from customization. Second, customization performed by one application should not affect other applications. This is critical since a custom mechanism that optimizes performance for one workload can reduce performance for other workloads.

Similar to other systems, userspace NVM file systems should ideally allow for workload-specific customization. However, as detailed below, existing userspace NVM file systems cannot fully achieve the above two characteristics.

2.3 Userspace NVM File Systems

Prior research on userspace NVM file systems has deeply investigated the direct access and customization benefits. In terms of direct access, prior work has proposed novel designs to enable userspace NVM file systems to directly access data [48] or metadata [38, 77]. In terms of customization, Aerie [68] pioneers userspace NVM file system customization and proposes flexible interfaces to develop LibFSes with high velocity. In addition, existing works on userspace NVM file systems also explore leveraging NVM in tired storage [53], maximizing multicore scalability [33], or minimizing indexing overhead [56].

Following the above discussion, to ensure security, the design of userspace NVM file systems focuses on preventing metadata (*i.e.*, any state other than actual data in regular files) corruption from malicious applications and LibFSes. This is because, thanks to page tables, userspace NVM file systems can offer the same security guarantee as kernel file systems to data in regular files. Unfortunately, despite their impressive achievements, existing userspace NVM file systems either enforce metadata integrity at a high performance cost (§2.3.1) or relax the security guarantee, leading to vulnerabilities (§2.3.2).

2.3.1 NVM FS based on metadata update mediation. One type of userspace file systems achieves metadata integrity by preventing LibFSes from directly updating metadata [33, 48, 53, 68]. Instead, a trusted entity, either in the kernel or as a privileged process, receives metadata update requests from a LibFS, validates the requests, and performs the update for the LibFS. This design introduces an undesirable coupling between file system design and the trusted entity. As a result, such a design significantly reduces performance, as we detail below.

Increase software overhead for metadata updates . One factor that reduces performance is that the trusted entity needs to mediate every metadata update. Therefore, all metadata updates involve expensive IPCs or context switches between the LibFS and the trusted entity, thereby significantly increasing the software overhead. ZoFS [38] reports that such mediation incurs an overhead of 44% for 4KB appends and 68% for file creations. Furthermore, some design allows LibFSes to read metadata directly, thus synchronizing metadata read from LibFSes and the metadata updates from the trusted entity becomes complicated, and is prone to become a scalability bottleneck. Existing designs rely on complex lock-free algorithms to resolve this challenge [33], but designing a completely scalable algorithms remains a challenge.

Aerie [68] and Strata [53] alleviate the communication overhead between LibFSes and the trusted entity by batching metadata update requests from a LibFS in a local log and later sending the log to the trusted entity. This design still suffers from software overhead, since there is an extra write to the log. Furthermore, the log also requires garbage collection mechanisms, leading to additional overhead and complexity. Finally, a log shared by all threads may become a scalability bottleneck, while a per-thread log requires a complicated algorithm for correctness [31].

Hinder customization. Another factor that reduces performance caused by mediating metadata updates is that such a design hinders customization by failing to meet the two requirements for customization discussed in §2.2. Specifically, with the aforementioned design, file systems bake the implementation of metadata updates into the trusted entity. Thus, for example, to customize certain file system data structures used in metadata updates (for example, the index structure that maps file offset to the underlying device blocks), an application needs special privileges to change the trusted entity. Furthermore, such changes affect all applications that share the trusted entity.

2.3.2 NVM FS based on direct metadata updates. Another type of userspace file systems [38, 56, 77]. design allows LibFSes to directly update metadata but at the cost of a relaxed guarantee on metadata integrity. For

example, ctFS [56] does not enforce metadata integrity, assuming that all applications sharing the file system trust each other. MadFS [77] allows direct updates to certain metadata in regular files and thus similarly suffers from some of the vulnerabilities mentioned below.

ZoFS overview. ZoFS allows direct metadata updates and offers protections with the coffer abstraction [38]. A coffer contains multiple NVM pages with the same access permission. ZoFS couples a LibFS to a coffer instead of an application as in prior designs (§2.3.1). Hence, applications access a shared coffer through the same LibFS and share the state of that LibFS (Figure 1). ZoFS does not prevent metadata corruption but instead mitigates its effects. For example, ZoFS uses Intel MPK [15] to make LibFSes access only one coffer at a time, thereby confining the metadata corruption within that coffer.

ZoFS limitations. A limitation of ZoFS is on violating one of the desired characteristics for customization (§2.2). Specifically, since multiple applications use the same LibFS to access one coffer, customizing one LibFS affects all the applications sharing that coffer.

Furthermore, ZoFS does not enforce metadata integrity and allows applications to update the shared coffer and LibFS state simultaneously. Although ZoFS can prevent some attacks with coffers and MPK, the aforementioned two design choices lead to several vulnerabilities. Some examples include (1) *Memory-based exploitation*. A malicious application (attacker) can modify pointers in file system data structures, causing another application (victim) to leak or overwrite sensitive information in DRAM. For example, suppose a victim copies a file. The attacker can modify the pointers in the source file’s indexes to point to the victim’s sensitive DRAM data, making the victim write it to the destination file. (2) *Denial of service attack*. For example, as noted by ZoFS’s authors, an attacker can hold locks in the LibFS forever. (3) *Semantic attack*. An attacker can perform attacks by violating file system guarantees. For example, an attacker can remove non-empty directories, making files disconnected from the root path. Other examples include creating files with the same name under one directory or causing loops in directory paths.

2.4 Lessons Learned from Prior Designs

Avoid metadata update mediation. Mediating metadata updates incurs significant overhead, hinders customization, introduces scalability bottlenecks, and greatly complicates system design (§2.3.1).

Enforce metadata integrity with trusted entities. Not enforcing metadata integrity leads to severe security vulnerabilities (§2.3.2). While it is possible to let a LibFS check the metadata integrity before its access, this imposes a significant burden on programming LibFSes. Furthermore, LibFSes do not have the knowledge and the privilege to handle corruption. For example, a reasonable policy to handle metadata corruption is to revoke access permission from a malicious or buggy LibFS. However, one cannot grant a LibFS the capability to revoke permissions from other LibFS. Thus, central trusted entities should enforce metadata integrity.

No simultaneous state sharing among applications. Sharing LibFS state among applications prevents per-application customization and incurs security vulnerabilities (§2.3.2). Thus, each LibFS should belong to only one application. Concurrent writes to NVM pages (e.g., coffer) from multiple applications also open up opportunities for attacks. Thus, as prior designs (§2.3.1), only concurrent read access or exclusive write access to a file should be allowed.

3 The TRIO Architecture

Motivated by the limitations of existing userspace NVM file systems and the lessons we learnt from them (§2), this section presents TRIO, a userspace file system architecture that unleashes the performance potential of NVM while ensuring metadata integrity. This section presents TRIO’s design goals and challenges (§3.1), design overview (§3.2), and concludes with a discussion (§3.3).

3.1 TRIO Design Goals and Challenges

We design TRIO to meet the following goals and resolve the inherent tensions among these goals.

- **Direct access.** To minimize software overhead and avoid scalability bottlenecks, TRIO allows a LibFS to access NVM directly to perform both data and metadata operations on both regular files and directories.
- **Unprivileged private customization.** To facilitate *unprivileged* customization, TRIO must cleanly decouple file system design from trusted entities; file system design is only LibFSes' responsibility. In this way, one can freely customize LibFSes without special privileges. Furthermore, a LibFS should not be shared so that applications can customize their LibFSes without affecting others.
- **File sharing.** Realizing the above two design goals unleashes the NVM performance potential for a single application. For multiple applications, to maintain the conventional file system abstraction, TRIO should allow multiple applications, each has a different private LibFS, to share files. However, with unprivileged private customization, each LibFS has its own file system semantics with potentially different data structures. Enabling *file sharing* among LibFSes in this case is challenging.
- **Metadata Integrity.** Since applications can be buggy or malicious, as with kernel file systems, TRIO must prohibit ill-behaved applications from (1) accessing data without permission and (2) conducting attacks on others by corrupting the file system's metadata.

It is advantageous for trusted entities to enforce metadata integrity (§2.4). However, TRIO's other design goals make it challenging. First, to meet the *direct access* design goal, trusted entities cannot use the conventional approach to mediate metadata updates, and thus are unaware of NVM state changes. Second, due to the *unprivileged application-level customization* design goal, the trusted entities do not understand LibFSes' data structures and thus cannot verify their integrity.

3.2 TRIO Overview

File system state separation. To overcome TRIO's design challenges, we consider the state in a file system. We find that it can be separated into (1) "core state", which is the essential state of a file system and cannot be generated if lost and (2) "auxiliary state", which can be discarded and if necessary, rebuilt from the core state. Hence, TRIO can use core state⁴ as common knowledge among components for sharing and integrity enforcement while achieving its performance goals with auxiliary state.

More specifically, core state contains the most important file system state: contents of files and the critical metadata to realize the basic file system abstraction (e.g., file names, file access permissions, the directory hierarchy). Auxiliary state contains the information to maintain file system interfaces (e.g., file descriptors), to achieve supporting functionalities (e.g., locks), and to speed up accessing core state (e.g., various types of caches, the inode bitmap). Core state must be in non-volatile storage since it cannot be lost, while auxiliary state can be in either non-volatile or volatile storage. The exact core state and auxiliary state depend on the file system design. We provide one example of POSIX-like file systems in §4.

With the state separation, to achieve its design goals, TRIO first explicitly defines the data layout and the data structure of a core state. All the components of TRIO, namely all LibFSes and the trusted entities, share this core state. A LibFS cannot change the data structure of the core state. Instead, each LibFS manages its private auxiliary state and can freely change such state. As detailed below, such state separation enables TRIO to overcome its design challenges and simultaneously meet all the design goals.

TRIO components. Figure 2 shows the components of TRIO and its workflow. Unlike prior architectures [33, 38, 48, 53, 68], thanks to the state separation, TRIO cleanly decouples file system design, access control, and integrity verification from each other. Specifically, TRIO consists of three (types of) components: per-application LibFSes,

⁴We note that the notion of "core state" and "auxiliary state" is similar to "hard state" and "soft state" discussed in prior work [64]

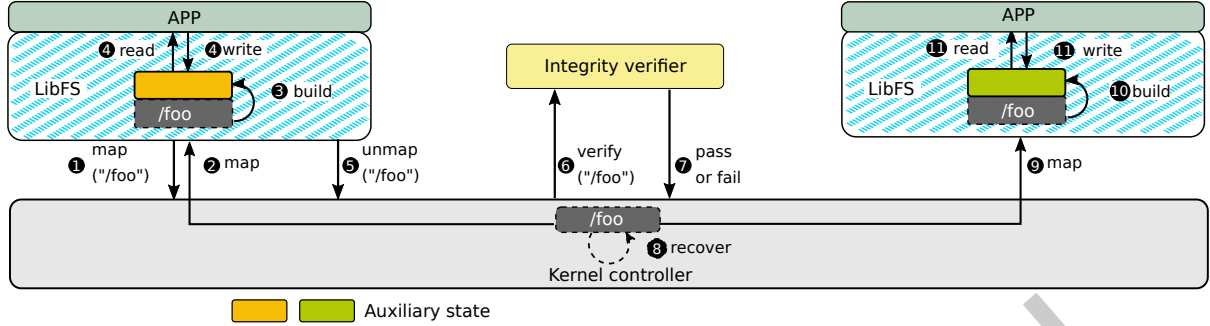


Fig. 2. TRIO enables file sharing between two different LibFSes. ① A LibFS requests access to a file, and ② the kernel controller grants it access to the file's core state. ③ The LibFS builds the file's auxiliary state. ④ Afterwards, the LibFS directly accesses the file. ⑤ Upon sharing, the LibFS unmaps the file, and ⑥ the kernel controller sends the file for verification. ⑦ The integrity verifier performs the verification. ⑧ If the verification fails, the kernel controller handles the corruption (§4.4). ⑨ Otherwise, the kernel controller grants the other LibFS access to the file's valid core state. ⑩ The LibFS builds the file's auxiliary state and ⑪ accesses the file.

an in-kernel access controller, and a trusted userspace integrity verifier as a standalone privileged process. Each LibFS has its own file system design based on its private auxiliary state. The kernel controller enforces each LibFS' access to the shared file system resources (e.g., NVM pages, inodes). More specifically, the kernel controller grants LibFSes access to the allocated resources (e.g., granting an LibFS access to a file) and allocating unused resources to LibFSes. Upon sharing a file, the integrity verifier inspects the file's core state modified by a LibFS (or a malicious application) to enforce metadata integrity.

Protected direct access. TRIO enables direct accessing with protection as follows. With TRIO, a file's NVM pages only contain the state of that file. When a LibFS accesses a file for the first time, it requests the kernel controller access to the core state of the file. If the LibFS has permission, the kernel controller grants access by programming the MMU. After obtaining access, a LibFS can access the core state for both metadata and data operations without involving any trusted entity, thereby achieving the *direct access* goal.

Flexible unprivileged private customization. Thanks to the state separation, file system design is solely the responsibility of LibFSes, where each LibFS maintains its own auxiliary state. Furthermore, each LibFS only belongs to one application. An application can thus freely perform customizations on LibFSes' by modifying their auxiliary state, thereby achieving the *unprivileged private customization* design goal (§5).

File sharing among different LibFSes. The sharing granularity in TRIO is a single file. Since every LibFS knows the data structure of core state, despite having a different design with e.g., different data structures, each LibFS can share a file by building its auxiliary state from the core state. As detailed below, TRIO enforces a concurrent reads or exclusive write file sharing policy. Hence, a LibFS does not need to handle stale auxiliary state; either the LibFS is the only writer, or the file is read-only.

Enforcing metadata integrity. Enforcing *metadata integrity* requires (1) avoiding simultaneous sharing of LibFSes and files state among applications; and (2) preventing metadata corruption (§2.4). To avoid simultaneous state sharing, TRIO associates a LibFS to one application and enforces concurrent read accesses or exclusive write access to a file. In other words, the kernel controller allows only one application to update a file at a time, while multiple applications can only readily-shared the file.

TRIO offers a guarantee that metadata corruption is confined to the application that causes it. As a result, TRIO prevents attacks from malicious applications while enabling direct access. To realize this guarantee, leveraging that only one LibFS can update a file at a time, when one LibFS releases its write access to a file, TRIO informs the

integrity verifier to check the file's core state. The LibFS then waits for the verification (and the potential fix for state corruption) to complete. If the verification fails, the integrity verifier informs the kernel controller to handle it. Section 4.4 details one checkpoint-based policy that also minimizes data loss by allowing LibFSes to *commit* changes.

Protecting from other security issues. TRIO prevents a LibFS (or an application) from using all resources (*e.g.*, inodes and NVM pages) in the file system. This is achieved by, as discussed earlier, making the kernel controller control the resource allocation; a LibFS requests the kernel controller to allocate more file system resources when it runs out of its own. Thus, kernel controller can audit the amount of resources each application spend and therefore limits resources allocated to the LibFS. In addition, to avoid one LibFS from holding the access permission of a file forever, kernel controller limits the amount of time one LibFS can access a file and revokes the access if the time exceeds.

Controlling the trust boundary. With TRIO, sharing a file across the trust boundary incurs the overhead of file mapping and unmapping, integrity verification, and rebuilding the auxiliary state. Following the conventional OS design, the default trust boundary in TRIO is a process. However, such a trust boundary is often too restrictive since processes may mutually trust each other (*e.g.*, processes belonging to the same application). TRIO's design enables a user to control the trust boundary explicitly, thereby improving sharing performance. Specifically, TRIO provides an abstraction named trust group to allow users control the trust boundary explicitly. Each trust group contains multiple processes belonging to the same user and mutually trusting each other. Thus, processes in the same trust group can share files with a shared LibFS and thereby avoiding the sharing overhead.

3.3 Discussion and Limitations

Data integrity. In essence, TRIO aims at offering the performance advantages of userspace file systems (by maximizing *direct access* and *unprivileged private customization*) while preserving the same abstraction (by enabling *file sharing*) and security guarantee (by enforcing *metadata integrity*) of kernel file systems. Therefore, enforcing data integrity (*i.e.*, protecting data in regular files) is orthogonal to TRIO's design. Since TRIO already enforces file access permissions (with MMU as discussed in §3.2), as with kernel file systems, the user is responsible for managing file access permissions to protect against malicious users. For unintentional corruptions caused by, *e.g.*, software bugs or hardware errors, file systems based on ARCKFS can design its core state to employ the relevant protection techniques.

Limitations. TRIO inherits two general limitations from prior userspace NVM file systems [33, 48, 53, 68]. First, file systems based on the TRIO architecture cannot fully conform to certain file system semantics like POSIX. Specifically, to ensure metadata integrity, TRIO cannot support concurrent write accesses from multiple untrusted applications to a file. Other examples include MMU cannot enforce the search permission of a directory, and file timestamps under certain scenarios cannot be precisely maintained (*e.g.*, the access time of a read-only file). Second, TRIO incurs extra costs upon file sharing. However, the sharing cost is not incurred when the file is read shared, or shared within a process or a trust group. The sharing cost is high only when multiple untrusted applications frequently write to a shared file. File systems in a central trusted entity are more suitable for the above scenario.

4 ARCKFS: POSIX-like FS Using TRIO

This section concretizes the design of TRIO by presenting ARCKFS, a userspace file system using the TRIO architecture. ARCKFS provides the POSIX APIs with similar file system semantics. The rest of the section presents ARCKFS's design goals (§4.1), its core state (§4.2), how it achieves high performance and multicore scalability (§4.3), enforces metadata integrity (§4.4), and preserves crash consistency (§4.5).

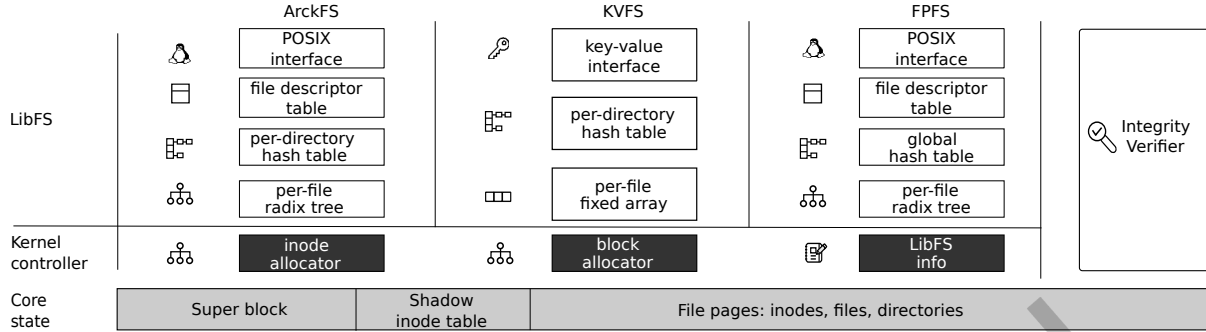


Fig. 3. An overview of the three presented userspace NVM file systems using the TRIO architecture. All the LibFSes share the same core state, kernel controller, and integrity verifier.

4.1 Design Goals

We design ARCKFS with the following goals to maximize the performance benefits enabled by TRIO (§3.1):

- **Minimal core state.** Deciding the core state of a file system involves tradeoffs. A large core state reduces the building time of the auxiliary state upon sharing and simplifies LibFS development. A small core state increases the flexibility of customization simplifies the integrity verifier, and reduces the time in verifying metadata integrity. ARCKFS chooses a minimal core state design to maximize performance for the common cases where file sharing (and the associated auxiliary state rebuilding) is rare.
- **Maximizing direct accesses.** To minimize software overhead, ARCKFS handles almost all the file system operations directly in its LibFS, exploiting direct NVM access enabled by TRIO. Only operations that involve permission changes, *i.e.*, `chmod` and `chown`, require trapping to the kernel controller.
- **Multicore scalability.** Modern servers have hundreds of CPUs. While NVM allows a high degree of concurrent access, storage stacks, especially the one in the Linux kernel, often introduces scalability bottlenecks (*e.g.*, VFS [33, 58]). TRIO enables secure and direct access, bypassing the kernel and the associated scalability bottleneck. Building upon this, ARCKFS further maximizes concurrent access with fine-grained parallelism.

4.2 Core state

Layout. As shown in Figure 3, ARCKFS’s core state consists of a superblock, a shadow inode table, and file pages. The superblock records the general file system information (*e.g.*, the total number of blocks). The kernel controller maintains the shadow inode table to enforce metadata integrity (§4.3). As detailed below, the file pages contain the inodes and data of directories and regular files. A LibFS has read access to the superblock, has no access to the shadow inode table, and the file access permission decides a LibFS’ access to file pages.

Next, we present the data structure design in file pages to minimize the core state while maximizing direct accesses by enabling MMU to correctly enforce the access permission for common file system operations.

Core state of a regular file. Common operations on a regular file are read, write, and truncate. Read requires file read permission, while write and truncate require write permission. Thus, to support direct userspace handling, a regular file contains index pages and data pages (Figure 4). Each entry of index pages points to a data page. The last entry of an index page points to the next index page. A special value (-1) marks the end of index pages. By mapping the index pages and data pages, the LibFS can directly handle read, write, and truncate. The LibFS persists data operations immediately (§4.5) and thus ignores `fsync()`.

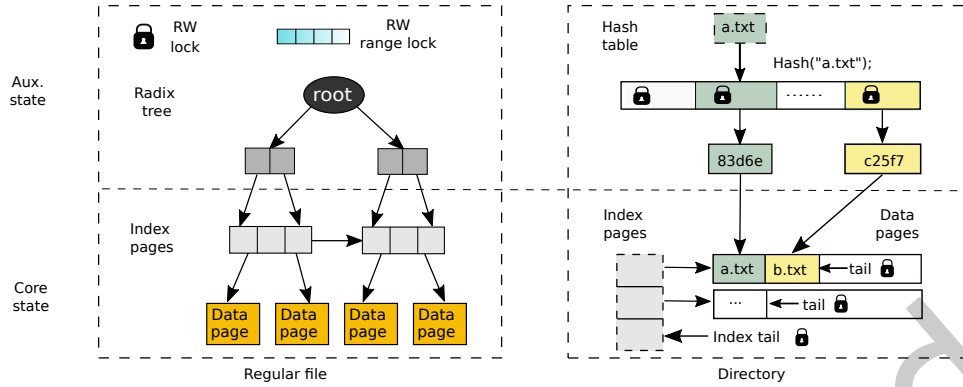


Fig. 4. ARCKFS's regular file and directory data structures (§4.3).

Core state of a directory. Common operations on a directory are search a file under it, list all files under it, stat (obtain the statistics of) a file, insert a file, and delete a file. Search, list, and stat operations require directory read permission and insert and delete require write permission. ARCKFS treats search permission the same as read permission since existing MMU cannot enforce search permission.

A directory contains index pages and data pages with directory entries. A directory entry contains information like inode number, file name, and the length of name. Thus, the LibFS can directly perform list and search by mapping the index and data pages of directories. As a result, LibFS resolves a path by mapping each directory's pages along the path and search for the next file.

As discussed above, stat, create, and delete require the read or write permission of the file's parent directory (instead of the file itself). To support direct userspace handling of these operations through page mapping, ARCKFS co-locates file's inode together with its directory entry. For example, in Figure 4, the "a.txt" (and "b.txt") block contains both the file's inode and its directory entry. Due to the co-location, there can be no "." and ".." in the core state of a directory. Instead, a LibFS maintains them in its auxiliary state. Section 4.4 discusses how to preserve the integrity of sensitive information in inodes (i.e., file access permission).

Discussion. We note two things in the above design. First, the data structures are also optimized for iteration: one can easily go over index pages to iterate the whole file. This is optimization is necessary since LibFSes and integrity verifier requires iterating the file data structures to build auxiliary state and check metadata integrity, respectively. Second, to simplify core state, both regular files and directories use the same data structures with indexing pages and data pages.

4.3 Handling File System Operations with LibFS

ARCKFS' LibFS handles file system operations by directly accessing its auxiliary state and the mapped core state (i.e., files' index and data pages). In addition, occasionally, the LibFS issues request to the kernel controller to, e.g., map or unmap files and allocate or free NVM pages (§4.6). We next discuss the design of LibFSes' auxiliary state (as shown in Figure 4) to enable efficient and scalable file system operations.

Single-threaded regular file operations. For a regular file, the LibFS uses a radix tree to map the offset within a file to the corresponding index page. Read, write, and truncate operations first walk the radix tree to find the data page and then performs the operations correspondingly.

Multi-threaded regular file operations. Inspired by prior works [33, 65, 78], LibFS enables fine-grained concurrent access to a file with a readers-writer inode lock and a readers-writer range lock. Thus, this design allows one thread to append or truncate the file (by acquiring the writer side of the inode lock), concurrent writes

to the disjoint region of the file (by acquiring the reader side of the inode lock and the writer side of the range lock), and concurrent reads to the file (by acquiring the reader side of the inode lock and the reader side of the range lock),

Data structure for directory operations. For each directory, the LibFS uses a resizable chained hash table to map the name of a file to its directory entry. The LibFS resizes the hash table when the entries to buckets ratio is greater or lesser than a threshold. To handle size increases, the LibFS maintains the tail of the index pages (index tail).

In addition, the LibFS extends the per-inode log design in NOVA [71, 72] by maintaining multiple logging tails. We note that, unlike conventional log-structured file systems [66], our design logs directory entries rather than operations. Multiple logging tails enables threads to create files in parallel, as detailed next. Our current implementation assigns each CPU with a private logging tail; other policies are also possible.

Each directory has three types of locks: per-bucket readers-writer locks in the hash table, a lock for each logging tail, and a lock for the index tail. As shown in §6.4, such a design achieves much better scalability than prior works.

Directory operations. The per-bucket locks are the main synchronization mechanism for directory operations by serializing operations involving the same directory entry. With these locks, ARCKFS ensures a consistent directory state, as detailed next.

Search and stat search the hash table and, if found, return the relevant information of the directory entry. List iterates the directory through the hash table rather than the directory's data pages. These operations only acquire the reader side of the per-bucket lock.

Insert first acquires the writer side of the per-bucket lock and inserts the entry in the hash table. Then, the thread checks whether the current CPU has been assigned a logging tail. If not, it acquires the index tail lock, allocates a new page, updates the index tail, and makes the logging tail point to the new page. Finally, the thread acquires the logging tail's lock, inserts the directory entry for the logging tail, and updates the logging tail.

Delete acquires the writer side of the per-bucket lock and accesses the hash table, marks the directory entry as deleted in its inode, and removes the corresponding hash table entry. The last step is necessary to avoid accessing deleted entries. Rename is performed as one insert and one delete operation. The atomicity is achieved by acquiring all the required per-bucket locks.

Permission change operations. LibFS must not execute operations (*i.e.*, `chmod` and `chown`) that change a file's access permission. With ARCKFS, the kernel controller executes these operations by maintaining file access permission in the shadow inode table. An LibFS traps in kernel controller to execute the permission change operations. As discussed in §4.1, a LibFS have direct access to the file's inode, which consists of its access permission. Thus, kernel controller notifies all the LibFSes, which have mapped the file's inode, about the permission change. An LibFS can also corrupt the permission information in the inode. Thus, when a file is unmapped, kernel controller overwrites the inode's permission information with the one in shadow inode.

Building auxiliary state from core state. For a regular file, the LibFS initializes a radix tree and iterates through the index pages to insert its address into the radix tree. It finishes the building by initializing the inode lock and the range lock. For a directory, the LibFS iterates the directory entries and inserts them into the hash table. It finishes the building by initializing the logging tails, the index tail, and their locks. A LibFS can preserve the auxiliary state of a file until another application requests to write to the file.

4.4 Enforcing Metadata Integrity upon Sharing

TRIO enables confining metadata corruption within the application that causes it (§3.2). This subsection presents how ARCKFS enforces this guarantee by detecting and fixing metadata corruption in a file's core state upon sharing.

Invariant I1	9 checks
5 checks	<i>Inode field checks:</i> Check inode fields.
3 checks	<i>Direntry field checks:</i> Check direntry fields.
1 check	<i>Direntry duplicity check:</i> No two direntries has the same name.
Invariant I2	6 checks
2 check	<i>Data page checks:</i> All the data pages are correctly allocated and unique.
2 check	<i>Index page checks:</i> All the index pages are correctly allocated and unique.
2 check	<i>Inode number checks:</i> The inode is correctly allocated and unique.
Invariant I3	2 check
1 check	<i>No disconnected inode check.</i>
1 check	<i>Root inode must be a directory</i>
Invariant I4	3 checks
3 checks	<i>Inode permissions are consistent with shadow inode.</i>

Table 2. Integrity checks categorized by file system invariants.

Detecting metadata corruption. The integrity verifier in ARCKFS is a trusted standalone process responsible for detecting metadata corruption. We refer to the integrity checks in the ext4 file system checker (*i.e.*, `e2fsck`) [10, 20] and adapt them to ARCKFS. Specifically, we derive our checks by going through the list of *all* possible errors `e2fsck` can report (`e2fsprogs/e2fsck/problem.h`). We also refer to the checks of metadata operations in the Linux kernel. For each of our check, we implemented unit tests by creating a corrupted instance of the file system and tested the implementation of the verifier.

We find that ARCKFS’ minimal core state greatly simplifies the integrity check. Also, the checks in ARCKFS are quite different from `e2fsck` and include a few extra checks due to (1) the differences in file system data structures and (2) that `e2fsck` checks file system integrity globally and offline while ARCKFS’ integrity verifier operates on an individual file and online. The integrity verifier detects metadata corruption by checking the following invariances shown in Table 2.

I1: Fields in each inode and directory entry are valid. The integrity verifier checks for invalid values of each field and inconsistency among fields within the file. Examples include (1) the file type is valid: either a directory or a regular file, (2) no file shares the same name under one directory, and (3) no “/” in file names.

I2: A file’s inode number, index pages and data pages are valid. Specifically, a file’s inode, index page, and data pages must either already exist before mapping to the LibFS or be allocated to the LibFS by the kernel controller. Furthermore, the inode and pages are not doubly referenced.

To perform these checks, the kernel controller maintains the following global file system information: (1) all the inodes and pages that are write-mapped or allocated to each LibFS and (2) all the inodes and pages that are in the existing files. The kernel controller updates such information when a LibFS maps or unmaps a file and allocates or frees inodes or data pages. The integrity verifier has read access to such information and can thereby perform the checks.

I3: The directory hierarchy forms a connected tree. Since ARCKFS’s directory core state does not contain “.” and “..” (§4.2), the integrity verifier is freed from checking the inconsistency caused by them. In addition, since ARCKFS currently does not support hard links, enforcing *I2* already prohibits cycles in the directory hierarchy. Thus, enforcing *I3* means the integrity verifier only needs to ensure the directory tree is connected. To achieve this, the integrity verifier compares the directory under check against its checkpoint state (used for recovery as detailed below) to identify *deleted* child directories. The integrity verifier then checks that the deleted child directory is not mapped to any LibFS and has no file under it.

I4: The access permission is correctly enforced. ARCKFS uses MMU to enforce the access permission under most scenarios, with one case requiring extra handling. Specifically, inodes mapped to LibFSes contain access permissions (§4.2). Therefore, any application with write access to the directory can modify the permissions, leading to security vulnerabilities. To resolve this, as discussed in §4.3, the kernel controller maintains file access

permission in the shadow inode table (§4.2) and uses them as ground truth; the ones in a normal inode are treated as cached state. To execute permission change operations (*i.e.*, `chmod` and `chown`), a LibFS issues request to the kernel controller to modify the access permission in the shadow inode.

A summary of integrity checks. Below, we summarize all the integrity checks the integrity verifier performs by categorizing them into checks in 1) directory entries, 2) inodes, and 3) the directory.

- Directory entry checks (6 checks)
 - `name_len` must be between 0 and 255.
 - `rec_len` is the sum of all field sizes
 - `name[]` does not contain “/”, is not “.”, and is not NULL
 - `ino_num` is not duplicated
 - `ino_num` must belong to this LibFS.
 - Two directory entries must not have the same name
- Inode checks (12 checks)
 - `file_type` must be either directory or regular file
 - `mode` - need to correspond to shadow inode
 - `uid` - need to correspond to shadow inode
 - `gid` - need to correspond to shadow inode
 - `size` - need to correspond to the number of index pages.
 - `atime` - between 1790.1.1 and 2310.04.04
 - `ctime` - between 1790.1.1 and 2310.04.04
 - `mtime` - between 1790.1.1 and 2310.04.04
 - All index pages must belong to this LibFS.
 - All index pages must not be doubly referenced.
 - All data pages must belong to this LibFS.
 - All data pages must not be doubly referenced.
- Directory checks (2 checks)
 - Root inode is a directory
 - No disconnected inodes

Fixing metadata corruption. TRIO can support multiple ways to fix metadata corruption and below presents one possibility. ARCKFS handles corruption by rolling back the file’s metadata to a checkpoint state while also including mechanisms to minimize data loss. Specifically, before the kernel controller grants one LibFS (LibFS A) write access to a file, it checkpoints the file’s metadata (*i.e.*, index pages for a regular file; both index and data pages for a directory). The kernel controller notifies the LibFS (LibFS B) that originally maps the file to discard the file’s auxiliary state and unmaps the file. Afterward, upon file sharing, the kernel controller notifies the integrity verifier to verify the integrity of the file’s core state. If the verification succeeds, the kernel controller grants LibFS A access to the file’s core state. Otherwise, ARCKFS notifies LibFS B to fix the corruption with a timeout. If LibFS B cannot fix the corruption, the kernel controller makes the corrupted file a private file to LibFS B and preserves LibFS B’s access to avoid data loss.

The kernel controller then fixes the corruption by making a copy of the corrupted file and reverts the file’s metadata state to a checkpoint state. An inconsistency might occur between the current and the checkpointed file size. The kernel controller resolves it by trimming or padding zero bits to the file. To further avoid data loss, the kernel controller provides a `commit` call for LibFSes to replace a file’s checkpoint with the current state, given that the current state passes the integrity check, thereby ensuring the kernel controller will not revert the changes.

Preventing the integrity verifier from being exploited. The integrity verifier is in a separate address space and we carefully design the integrity verifier to prevent it from being exploited. Our design follows the principle

of least privilege; the integrity verifier is in the userspace without root privilege and has only read access to only the required file system state. The integrity verifier forks itself to handle a new integrity check. We employ defensive programming in the integrity verifier.

4.5 Crash Consistency

Overview. ARCKFS splits the responsibilities between LibFSes and the kernel controller to achieve crash consistency. Specifically, ARCKFS’ core state (§4.2) does not specify a crash consistency mechanism to enable LibFSes to design their own, facilitating customization. However, when a LibFS registers with the kernel controller, it also needs to specify a program to handle crashes, since unlike a traditional in-kernel file system, kernel controller does not know how to recover from the crash. Therefore, upon reboot after a crash, the kernel controller invokes this program to perform recovery. Since ARCKFS cannot trust the programs provided by LibFSes, it invokes the integrity verifier after recovery to ensure the valid state of all files that are mapped as writable when the crash occurs.

Consistency mode of the LibFS. We next discuss the design of the crash consistency mechanism of ARCKFS’ LibFS. Similar to other designs [39, 47, 48, 71, 78], the LibFS ensures all metadata operations are synchronous (*i.e.*, the operation persists on NVM before the system call returns) and atomic (*i.e.*, no partial update). Data operations are synchronous but not atomic (*i.e.*, a partial write is possible upon crash). Extending the LibFS to support other consistency modes is simple by following the prior approaches [48, 71].

Consistency mechanism in the LibFS. Current hardware supports atomic NVM updates up to 16 bytes [39, 78]. For most operations, ARCKFS’s core state design (§4.2) enables LibFSes to use atomic updates to ensure crash consistency. For example, during file creation, LibFS first persists all other writes of the directory entry with an inode number 0 to mark the directory as invalid. Finally, it persists the update to the inode number. A few complex operations, such as rename, require journaling. ARCKFS uses undo logs for simplicity. LibFS uses its reserved pages for journaling (§4.6).

4.6 Implementation

We implement the kernel controller as a module for the 5.13.13 Linux kernel. As with prior works [53, 68], the kernel controller uses leases to prevent a LibFS from holding a file forever. If a LibFS holds the file for more than the allocated time, the kernel controller forcefully unmaps the file. The LibFS’ index structures, specifically the radix tree and the hash table (§4.3), are inspired by ScaleFS [31]. As with NOVA [71] and WineFS [47], the kernel controller and LibFS implement the heap and inode allocators in DRAM as red-black trees. However, since the aforementioned file systems are in the kernel, most of the time, we need to reimplement the data structures in the LibFS. We implement the kernel controller and the integrity verifier from scratch. The kernel controller, LibFS, and the integrity verifier have 3791, 7586, and 457 lines of code, respectively. The code of the integrity verifier is small since there is no complex data structure in ARCKFS’s core state, and verifying metadata integrity in runtime avoids many checks [42].

Multicore scalability. ARCKFS’s design enables fine-grained parallelism to directories and files (§4.3). In addition, we make key data structures in the kernel controller and LibFS per-CPU, including the block allocators, inode allocators, file descriptor allocators, and journal. ARCKFS implements its readers-writer locks based on state-of-the-art techniques [37].

Two-level allocation. ARCKFS uses a two-level allocation design for NVM pages and inodes for efficiency and security. The kernel controller allocates NVM pages and inodes in batch to LibFSes. Each LibFS has its own allocators. Importantly, this design enables the kernel controller to audit and restrict resource usage [16] for each application and its user.

Huge pages. ARCKFS currently uses 4KB pages for NVM. We do not see inherent challenges in supporting huge pages.

Overcoming the virtual memory bottleneck Linux’s virtual memory management is known to be a scalability bottleneck [34, 37, 52], which slows down operations like (un)mapping files with ARCKFS. To overcome this, upon LibFS initialization, the kernel controller creates a special VMA that is large enough to contain all NVM pages. The kernel controller maps NVM pages to the special VMA to avoid VMA layout change. For correctness, the special VMA does not support operations like on-demand paging. Therefore, this optimization removes the scalability bottleneck in VMAs.

NVM write protection. To prevent bugs from corrupting LibFS and NVM state, ARCKFS uses Intel memory protection key (MPK) [15] to place NVM, LibFS, and application into separate protection domains. A thread acquires write access to the NVM or LibFS protection domain immediately before the updates and releases it after the update completes.

Reserved page. An LibFS may require NVM pages to implement its own persist data structures (e.g., journaling). ARCKFS supports this with reserved pages. kernel controller maintains the metadata of the reserved pages in the LibFS descriptors (§4.2) and the integrity verifier does not check the integrity of a reserved page.

Adapting to Intel Optane PM. We implement ARCKFS for the Intel Optane PM since it is the only publicly available NVM. Due to the hardware design of Optane PM, excessive concurrent access from multiple CPUs and remote NUMA access significantly degrades performance [36, 45, 69, 73]. Thus, ARCKFS employs the opportunistic delegation technique in OdinFS [78] to maximize PM performance. Specifically, ARCKFS creates multiple background kernel threads (delegation threads) in each NUMA node. The delegation threads are shared by all LibFSes. Application threads cannot access NVM but instead sends the access request to one of the delegation threads in the corresponding NUMA node with a per-application ring buffer. Afterward, the application threads wait for the delegation threads to perform and complete the access. The fixed number of delegation threads avoids performance collapse due to concurrent access. Moreover, delegation threads always perform local NVM access. Therefore, it scales NVM performance. Furthermore, by striping the file data across NVM in multiple NUMA nodes, ARCKFS handles bulk data operations by letting delegation threads access NVM nodes in parallel, fully utilizing aggregated NVM bandwidth. Due to the communication overhead, ARCKFS does not delegate small NVM access (read access less than 32KB and write access less than 256 bytes).

4.7 Discussion

The scalability of the integrity verifier. ARCKFS currently serializes integrity checks by running integrity verifier as a single-threaded process. This is because, as discussed below, it is non-trivial to scale the integrity verifier, and the single-threaded implementation is sufficient for the target use case, where file sharing is rare.

Scaling the integrity verifier is challenging because its operations do not commute, so it cannot linearly scale with more threads, according to the scalable commutativity rule (SCR) [35]. Briefly, the SCR states that if an operation commutes with another operation (*i.e.*, changing their orders does not affect the outcome), there exists a scalable implementation of these operations. While the SCR does not explicitly state the reverse (*i.e.*, that if an operation does not commute with another, they cannot scale), it is a strong indication that non-commuting operations can hardly scale [30, 63]. The integrity checks are non-commuting with any modification performed by LibFSes. In addition, the checks do not commute with each other (e.g., if two LibFSes use the same inode number, the checks for the latter LibFS will fail). Therefore, we choose to serialize the integrity checks.

However, despite the challenges, we note that one can still increase the scalability of the integrity verifier. This is because similar problems have been observed and greatly advanced in other domains, such as database transactions and file system journaling. As with the verifier, a transaction or the sync() operation does not commute with any write modifications or with itself. Nevertheless, journaling and database transactions can still

achieve good scalability through clever techniques developed by prior works [41, 51, 57, 59, 60]. Inspired by this, we believe that there exists more scalable implementations of the integrity verifier, which we leave as future work.

Fairness. Two types of fairness are relevant to ARCKFS: 1) the amount of time each application can access a file, and 2) the portion of NVM bandwidth each application may use. For the former, as the trusted kernel controller controls file sharing, ARCKFS can enforce a wide range of policies. Our current implementation uses a simple FIFO policy with bounded access time (enforced with leases; see §4.6). Nevertheless, more advanced policies are possible. For example, the kernel controller may control access time based on priority and/or application type (e.g., ARCKFS may prioritize user-facing applications’ file access over batch-processing ones).

Enforcing the latter type of fairness remains an open problem, where the key challenge is that applications can directly access NVM. Prior work, such as MT^2 [75], has made promising progress, which we plan to explore in ARCKFS. Following MT^2 , ARCKFS can enforce fairness by combining both hardware and software techniques. Specifically, ARCKFS can use commodity hardware mechanisms, such as Intel MBA [13] and ARM MPAM [2], to limit the memory bandwidth an application can use. However, these mechanisms only limit the total memory bandwidth, which includes both NVM and DRAM accesses. Therefore, to further limit the NVM bandwidth, ARCKFS coordinates with the thread scheduler to preempt applications’ threads when it exceeds the allocated NVM bandwidth.

5 File System Customization

This section showcases TRIO’s unprivileged private customization advantage with two customized LibFSes: KVFS, which is very similar to the customization case in Aerie [68], and FPFS, which is based on full path indexing [67, 76]. KVFS and FPFS’s designs are based on ARCKFS’s core state (§4.2), involving heavy changes in the interfaces and metadata structures of ARCKFS’s LibFS. Furthermore, KVFS and FPFS assume specific workloads and cannot (efficiently) handle other workloads or even certain basic file system operations. For example, FPFS cannot efficiently handle rename. Nonetheless, realizing KVFS and FPFS with TRIO does not require modifying the trusted entities, and TRIO enables deploying them to only applications that can benefit without affecting other applications. Neither of these two advantages is possible with Aerie.

KVFS. Applications, such as email clients [6] and some HPC applications [18, 19], operate on many small files. A generic file system incurs the overhead of file descriptors with the open and close interfaces. Furthermore, the file system also suffers from the overhead of index structures. Specifically, upon file creation, deletion, and access the file system needs to create, delete, and walk the indexing structures of the file. Both overheads are too high for small files.

We design KVFS to optimize applications working with many small files. Specifically, we add to ARCKFS’ LibFS get and set interfaces, that directly read from, or create and write to a specified file, respectively. The get and set APIs always operate from the beginning of a file. Thus, it does not need to maintain file descriptors, thereby eliminating their overhead. KVFS assumes a small maximal file size (32KB in our design). Thus, we replace the radix tree in ARCKFS’ auxiliary state with a fix-sized array, thereby minimizing the overhead associated with index structures. Finally, with many files, concurrent accesses from threads to the same file becomes unlikely. Therefore, we replace the fine-grained locks in ARCKFS’ auxiliary state with a simple spinlock to optimize for non-contended cases.

FPFS. With a typical file system, resolving a path requires iterating through each directory along the path. This approach incurs high overhead with deep directory hierarchies. We design FPFS for applications working with deep directory hierarchies. FPFS replaces the per-directory hash table in ARCKFS’ auxiliary state with a global hash table that directly maps a path to the corresponding directory entry in the core state. It thus eliminates

- D. Zhou et al.

directory traversals and significantly improves the performance of applications working with deep directory hierarchies.

We note that, as mentioned earlier in this Section, full path indexing makes it extremely challenging to handle operations such as rename and directory list. Furthermore, with concurrent updates from different threads, the global hash table may become a scalability bottleneck. Fortunately, Trio's design enables deploying FDFS to only the applications that can benefit from it without affecting other applications.

Customization limits. To enforce metadata integrity, Trio limits the scope of customization to the auxiliary state in each LibFS (§3.2). Hence, customizations involving changing the core state require special privileges. For example, ARCKFS's core state cannot support conventional log-structured file systems [55, 66]. Nonetheless, ARCKFS's minimal core state (§4.2) can enable various customizations, including file system interfaces, caching mechanisms, key data structures, concurrency control, and crash consistency, which is more flexible than prior approaches like Aerie.

6 Evaluation

Our evaluation aims to reveal the performance and security of file systems based on Trio. Specifically, our evaluation answers the following questions:

- What is ARCKFS's performance with a single thread (§6.2) and multiple threads (§6.3)?
- Does ARCKFS scale file system operations? (§6.4)
- Can ARCKFS ensure metadata integrity upon malicious LibFSes and what is the sharing cost? (§6.5)
- How do ARCKFS and the customized file system perform with macro-benchmarks and LevelDB? (§6.6)

6.1 Evaluation setup

Environment. Our evaluation machine has eight sockets. Each socket equips with a 28-core Intel Xeon Platinum 8276L processor (with 38.5MB last level cache) and six Intel first generation Optane PM DIMMs (interleaved at 4KB). The server has 224 cores and 6144GB PM storage in total. The system runs Ubuntu 20.04 and Linux kernel 5.13.13 with hyperthreading and Turbo Boost disabled.

Baseline file systems. We compare ARCKFS with five in-kernel file systems: ext4 [7], PMFS [39], NOVA [71], OdinFS [78], WineFS [47], and two userspace file systems: SplitFS [48], Strata [53]. ext4 is a mature and widely used real-world file system and others are state-of-the-art NVM research file systems. PMFS is developed by Intel to exploit NVM characteristics, particularly the byte addressability. NOVA and WineFS improve upon PMFS with efficient data structures and faster crash consistency mechanisms (per-inode log in NOVA and per-CPU journaling in WineFS). OdinFS builds upon NOVA and WineFS with opportunistic delegation to maximize NVM performance (§4.6).

SplitFS and Strata are the only userspace file systems we can run and evaluate (as detailed below). Both handle data operations in userspace and require a trusted entity to handle metadata operations (§2.3.1). The default option for the trusted entity is the ext4 file system for SplitFS and a privileged process for Strata, respectively.

Configuration. We configure the baseline file systems with the default setup, except we enable the DAX option for ext4 to improve its performance on NVM. The baseline file systems provide weaker or the same crash consistency guarantee as ARCKFS (§4.5). We evaluate ARCKFS with two setups. The default one utilizes all eight NVM NUMA nodes, each with twelve delegation threads (following OdinFS' default setup). This setup demonstrates the maximal throughput and scalability ARCKFS can achieve. The direct comparison for ARCKFS, in this case, is 1) OdinFS, which we configure with the same setup (*i.e.*, eight NUMA nodes and twelve delegation threads); and 2) ext4(RAID0), where we further create a RAID0 of NVM nodes [5] and mount ext4 on top of it (ext4(RAID0)). We only use ext4 since other evaluated NVM file systems cannot operate on a Linux RAID.

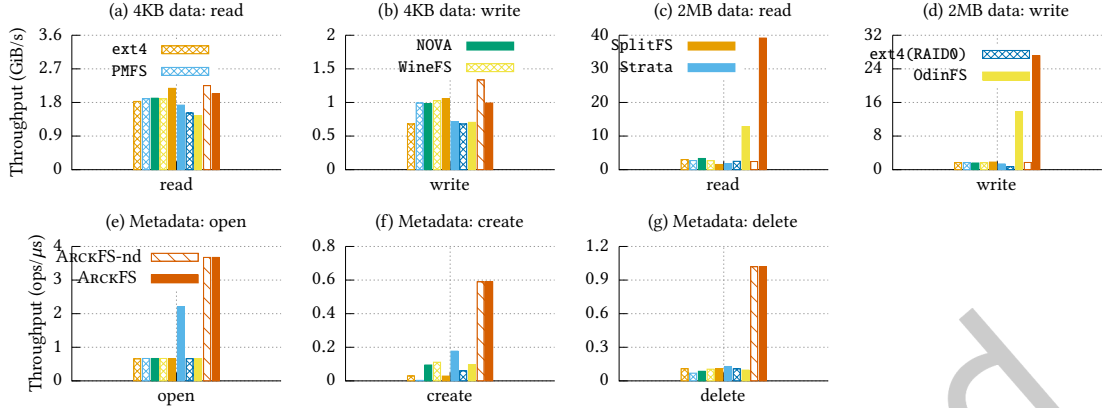


Fig. 5. Single thread performance of the evaluated file systems.

Finally, to compare ARCKFS with most NVM file systems that do not consider multiple NVM NUMA nodes and the Optane characteristics discussed in §4.6, we also evaluate ARCKFS on a single NUMA node without opportunistic delegation (ARCKFS-NO-DELE).

Workload. Our workloads cover a wide range of file system use cases. For microbenchmarks, we use the popular fio [9] and FxMARK [58] to evaluate latency, throughput, and multicore scalability, focusing on both data and metadata operations. We configure fio to let each thread randomly access a 1GB private file. We show fio’s random access results and sequential access yields similar results. We use Webserver, Fileserver, Webproxy, and Varmail in Filebench [8] and LevelDB as macrobenchmarks.

Limitations. We test ARCKFS’ crash consistency with unit tests during development but do not evaluate it with testing frameworks such as Chipmunk [54]. We cannot run Strata beyond one thread (and thus only show its results in §6.2). We do not include ZoFS [38] and KucoFS [33] due to no access to their source code. We are unable to run Aerie [68] and ctFS [56] with the provided configurations.

6.2 Single thread performance

Figure 5 shows the performance of common file system operations with a single thread. With the 4KB access size, SplitFS and ARCKFS-NO-DELE outperform NOVA by 9% – 31% due to direct NVM access. The performance difference between SplitFS and ARCKFS-NO-DELE is due to different implementations of memcopy. Opportunistic delegation introduces the overhead of striping data across NVM nodes and the communication overhead. As a result, ARCKFS is 21% slower than ARCKFS-NO-DELE but still outperforms NOVA by around 6%.

With the 2MB access size, the advantage of direct NVM access is minimal since the data copy time dominates. OdinFS and ARCKFS parallelize data access with opportunistic delegation and thus perform much better than others. In this case, since application threads only need to send requests to delegation threads (instead of performing data copy), avoiding kernel trapping brings a significant advantage. In summary, ARCKFS outperforms others by 3.1× to 25× and 2.0× to 15× for 2M-read and 2M-write, respectively.

Workloads for metadata performance include open and close a file in a five-depth directory, create an empty file, delete all the empty files under one directory. Since small NVM accesses are not delegated (§4.6), ARCKFS-NO-DELE performs similarly to ARCKFS for metadata operations. ARCKFS’s performance advantage comes from both direct NVM access and efficient data structures (§4.3). Taking create as an example, we find NOVA and Strata spend at least 42% and 44.5% of the time in VFS and digestion, respectively. Besides that, the rest of the performance difference is due to the data structure design. For example, with our workload, we find the index data structures in

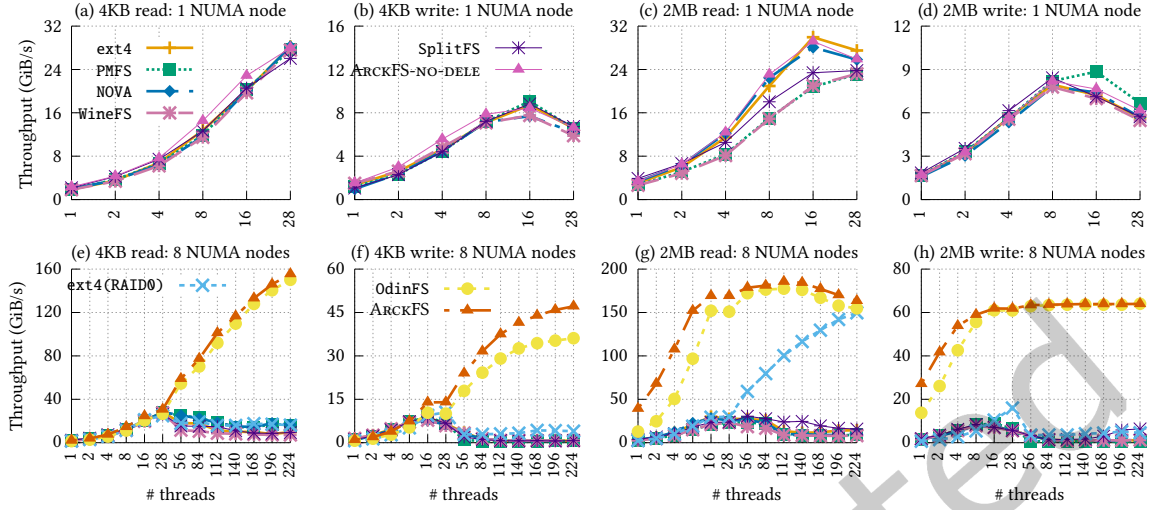


Fig. 6. Throughput of the evaluated file systems with one and eight NUMA nodes. ARCKFS follows OdinFS’s design to scale NVM performance and outperforms OdinFS with direct NVM access.

NOVA (radix tree) are slower than ARCKFS’s hash table. As a result, for open, create, delete ARCKFS outperforms others by 1.6× to 5.6×, 3.3× to 5.3×, and 7.4× to 9.4×, respectively.

Summary. ARCKFS outperforms due to direct access (for both data and metadata operations), opportunistic delegation (for data operations), and efficient data structures (for metadata operations).

6.3 Data operation performance

Figure 6 shows the throughput of each file system with one and eight NUMA nodes evaluated with fio. With one NUMA node, for 4KB read and write, ARCKFS-NO-DELE outperforms other file systems by 10% – 12% due to direct access. For 2MB read and write, all the evaluated file systems perform similarly. The throughput drop is due to excessive concurrent accesses (§4.6).

With eight NUMA nodes, for 4KB-read and 4KB-write, when the thread count is low, ARCKFS (and OdinFS) performs similarly or worse than other file systems (§6.2), due to the overhead of striping file data and delegation. However, OdinFS and ARCKFS can scale to 224 threads because they employ opportunistic delegation to preserve NVM performance (§4.6). ARCKFS further outperforms OdinFS due to direct NVM access. With 224 threads, ARCKFS outperforms OdinFS by up to 1.3× and other file systems by up to 22×.

For 2MB-read and 2MB-write, since OdinFS and ARCKFS parallelize NVM access (§6.2), they constantly outperform other file systems. ext4(RAID0) also scales 2MB-read because it reduces the degree of concurrent access to NVM and remote NVM reads incurs lower overhead than writes ([78]). ext4(RAID0) does not scale 4KB-read due to a scalability bottleneck. When the thread count increases, the NVM bandwidth becomes the performance bottleneck and OdinFS starts to catch up with ARCKFS. With 224 threads, ARCKFS outperforms evaluated file systems by 1.1× to 25×, and up to 15× for 2MB-read and 2MB-write, respectively.

Summary. ARCKFS follows OdinFS’s datapath design to scale NVM performance for both read and write operations. ARCKFS further outperforms OdinFS with direct NVM access.

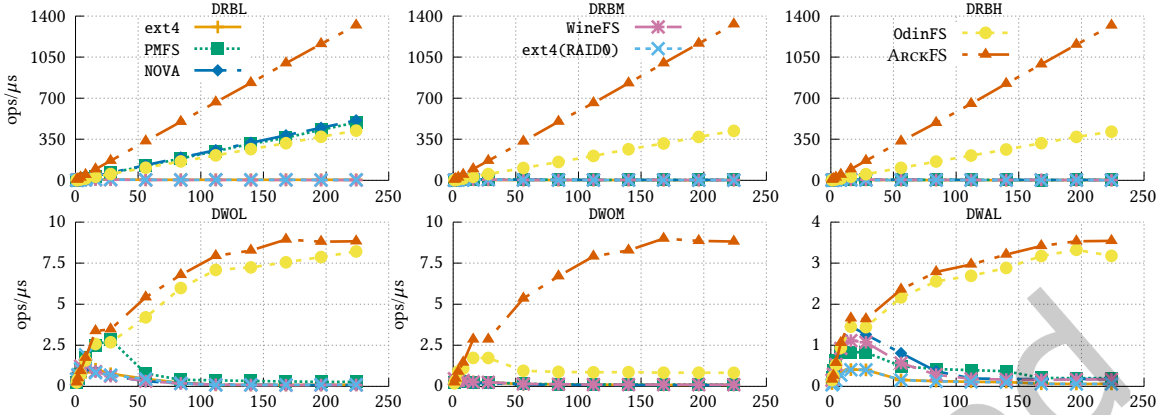


Fig. 7. Datapath scalability of the evaluated file systems.

6.4 Scalability

We evaluate file system scalability (*i.e.*, performance with increasing number of threads) using the FxMARK benchmark suite. Each benchmark in FxMARK creates multiple threads and each thread repeats the same operation. Table 4 summarizes FxMARK’s metadata benchmarks.

Data operation scalability. Table 3 summarizes FxMARK’s data microbenchmarks and Figure 7 shows the results. It does not include the results of DWSL since it is the same as DWOL; all evaluated file systems persist operations immediately and treats `fsync()` as no-ops.

For data operations, all the evaluated files bypass the kernel (or VFS) and thus are not affected by its scalability bottleneck. For read, ARCKFS and OdinFS scale all three microbenchmarks, whereas except them, only PMFS and NOVA only scale DRBL. ARCKFS and OdinFS scale because they enable fine-grained access (§4.3). Furthermore, they use advanced synchronization techniques (§4.6) to minimize the locking overhead, while for others, specifically with DRBM and DRBH, the standard locks in the kernel becomes a scalability overhead. ARCKFS outperforms OdinFS due to direct NVM access. The advantage is higher than that in §6.3 since the small file used in FxMARK are in the CPU cache during execution. This reduces file access time and leads to a higher kernel bypass advantage. In summary, ARCKFS scales linearly in DRBL, DRBM, and DRBH, with 224 threads, outperforming others by 2.5×, 3.2× to 730×, and 3.2× to 850×, respectively.

For write operations, only ARCKFS and OdinFS scale because they prevent NVM performance collapse with opportunistic delegation (§4.6). They scale DWAL with a scalable allocator design. ARCKFS scales DWOM by allowing writes to disjoint file regions (§4.3). With 224 threads, ARCKFS outperforms other by in DWOL (DWSL), DWOM, DWAL by 1.1× to 57.8×, 10.6× to 87×, and 1.1× to 9.1× respectively.

Summary. ARCKFS and OdinFS scales with opportunistic delegation, fine-grained file access, and scalable data structures. ARCKFS outperforms OdinFS with direct NVM access.

Metadata operation scalability. Figure 8 presents the scalability of metadata operations. Essentially, for all the other evaluated file systems, VFS decides their scalability. Specifically, most other file systems can only scale MRPL and MRDL since the scalability bottlenecks in VFS (*e.g.*, coarse locks on directory cache, inode cache, directory inode and the global rename lock) prevent scaling other microbenchmarks [58].

Direct NVM access allows ARCKFS to avoid the scalability bottleneck in the VFS. Furthermore, ARCKFS scales due to its scalable data structures and lock design (§4.3, §4.6). Hence, for DWTL and all the read-dominated

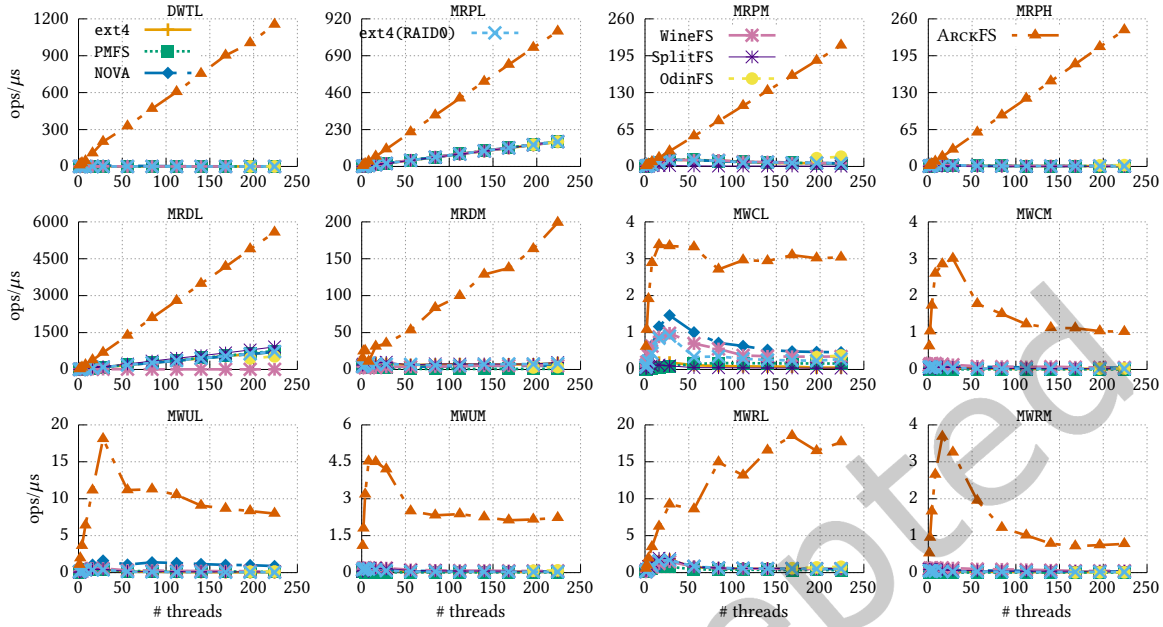


Fig. 8. Metadata scalability of the evaluated file systems. ArckFS scales thanks to kernel bypassing and scalable design. Results with one NUMA node are similar to those under 28 threads and thus omitted.

Name	Description
DRBL	Each thread reads a private page in a private file.
DRBM	Each thread reads a private page in a shared file.
DRBH	Each thread reads a shared page in a shared file.
DWOL	Each thread overwrites a private page in a private file
DWSL	Same as DWOL plus an <code>fsync()</code> after each write
DWAL	Each thread appends to a private file
DWOM	Each thread writes to a private page in a shared file

Table 3. Summary of FxMARK’s data microbenchmarks. During the execution, each thread of the benchmark repeats the corresponding operation in the table.

Name	Description
DWTL	Reduces the size of a private by 4K.
MRP(L/M/H)	Open a (private/random/same) file in five-depth dirs.
MRD(L/M)	Enumerate files of a (private/shared) directory.
MWC(L/M)	Create an empty file in a (private/shared) dir.
MWU(L/M)	Unlink an empty file in a (private/shared) dir.
MWRL	Rename a private file in a private dir.
MWRM	Move a private file to a shared dir.

Table 4. Summary of FxMARK’s metadata microbenchmarks. During the execution, each thread of the benchmark repeats the corresponding operation in the table.

workloads, ArckFS scales linearly. Specifically, for microbenchmarks performing open and enumerate, at 224 threads, ArckFS outperforms others by 5.4× to 334× and 7.4× to 25×, respectively.

For MWCL and MWUL, ArckFS does not scale linearly due to excessive concurrent NVM access; these small accesses are not delegated (§4.6). This does not affect MWRL since ArckFS writes much less NVM data for it. The scalability of MWCM, MWUM, and MWRM drops due to the contention in ArckFS’s directory hash tables. Furthermore, MWCM and MWRM also contend on logging tails and the index tail (§4.3). In summary, for microbenchmarks that perform create, unlink, and rename, at 224 threads, ArckFS outperforms by 2.3× to 21.2×, 8.9× to 32.7×, 16.2× to 36.4×, respectively.

Summary. Direct NVM access allows ArckFS to bypass the scalability bottleneck in VFS. Leveraging this, the careful data structure design in ArckFS makes it scale significantly better than existing NVM file systems.

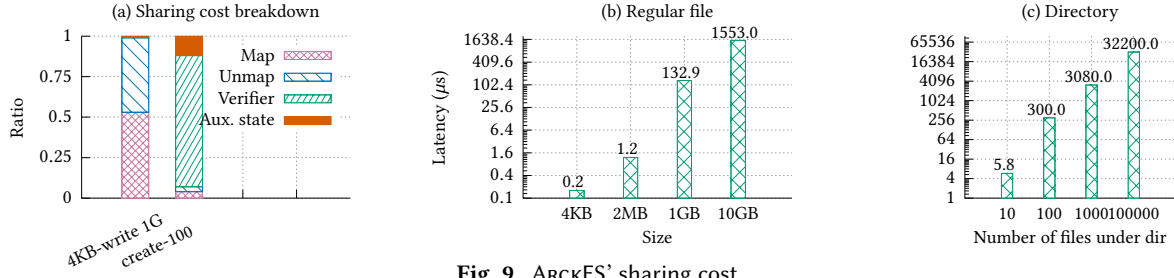


Fig. 9. ARCKFS' sharing cost.

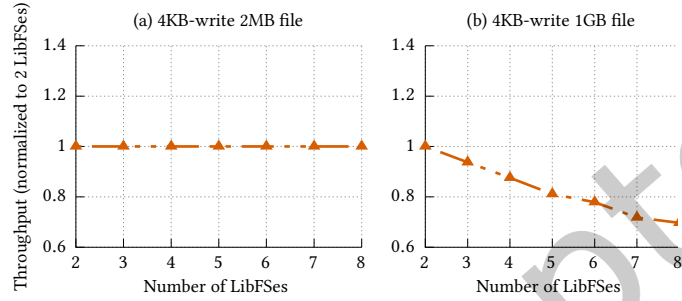


Fig. 10. How the verifier scales with the number of LibFSes.

6.5 Metadata Integrity and Sharing Cost

Detecting and recovering from metadata corruption. We design tests emulating both malicious LibFSes and buggy LibFSes to stress the metadata integrity enforcement design in ARCKFS (§4.4). Specifically, we handcrafted eleven attacks performed by a malicious LibFS corrupting metadata, some mentioned in §2.3.2. For example, the malicious LibFS (1) modifies pointers in index pages to point to DRAM data; (2) removes a non-empty directory; (3) creates file names containing “/” to trick another LibFS into accessing a wrong file; (4) causes loops within a file’s index pages.

To emulate a buggy LibFS, for each integrity checks in the verifier, we create an automated script to corrupt the relevant metadata with, say, a random value. We also run different scripts together to cause more complex corruption. In total, we cause 134 corruption scenarios. In all the test cases, the integrity verifier can detect the corruption, and the kernel controller can restore the corrupted file to a consistent state.

Sharing cost. When multiple untrusted applications concurrently update a file, ARCKFS incurs a sharing cost caused by file mapping and unmapping, integrity verification, and rebuilding the auxiliary state. We evaluate the sharing cost with (1) two applications writing 4KB to a 2MB (4KB-write 2MB) or a 1GB file (4KB-write 1GB) and (2) two applications creating empty files in a directory that contains 10 (create-10) or 100 files (create-100).

Table 5 shows the results. When the file/directory size is small, the sharing cost is negligible ($< 5\mu\text{s}$) for write and modest for create, leading to similar performance as NOVA. The overhead increases when the file/directory size grows. Figure 9(a) shows the breakdown of the overhead. Specifically, for 4KB-write 1GB, mapping and unmapping files, where each iteration takes 670ms on average, contribute to 99% of the overhead. With ARCKFS’s 100ms lease time, this results in an overhead of 7.8 $\times$. For create, we stress the sharing overhead by making applications unmap a file after each operation. Thus, in this case, the overhead of verification (300 μs , 81%), and rebuilding the auxiliary state dominates (12%), leading to an overhead of 20.4 $\times$. Leveraging the trust group (§3.2) can eliminate the overhead.

	NOVA	ARCKFS	ARCKFS-trust-group
4KB-write 2MB	1.92GiB/s	1.90GiB/s	1.95GiB/s
4KB-write 1GB	1.91GiB/s	0.25GiB/s	1.95GiB/s
Create 10	8.2 μ s	7.8 μ s	1.5 μ s
Create 100	8.4 μ s	32.7 μ s	1.6 μ s

Table 5. Performance of ARCKFS when two threads concurrently update the same file. In summary, trust group can prevent the sharing overhead.

Name	# Files	Avg. file size	I/O size (r/w)	R/W
Fileserver	10K	2MB	1MB / 512KB	1:2
Webserver	20K	4MB	1MB / 256KB	10:1
Webproxy	100K	512MB	1MB / 16KB	5:1
Varmail	100K	16KB	1MB / 16KB	1:1

Table 6. Filebench workloads configurations, which aim to cover a wide range of file system use cases.

Scalability of the verifier Figure 9(b) and (c) show how verification time changes with file and directory sizes. The verification time increases almost linearly as the file size grows. For a regular file, almost all time is spent verifying that it contains the right NVM pages. For a directory, over 90% of the time is to verify that files under the directory have different names.

Figure 10 shows how the performance under sharing changes with the number of LibFSes. For 2MB files, the performance remains similar as the number of LibFSes increases, since the verification time is negligible. For 1GB files, with 8 LibFSes and a single-threaded verifier (§4.7), the performance is around 70% of that with 2 LibFSes. We believe the performance drop is acceptable, as it is unlikely that many LibFSes will concurrently update a file.

Summary. ARCKFS can effectively enforce metadata integrity in the presence of buggy or malicious LibFSes. Concurrent write access to a shared file from untrusted applications incur sharing cost for ARCKFS. If applicable, a user can use the trust group to avoid it.

6.6 Macrobenchmarks and Real-World Applications

Filebench. We use four Filebench benchmarks: Fileserver, Webserver, Webproxy, and Varmail with configurations shown in Table 6. We aim to cover a wide range of file system use cases with such configurations. Specifically, Fileserver and Webserver are data-intensive, performing large file writes and reads, respectively. Webproxy stresses both data and metadata operations, performing small file reads. Varmail is metadata-intensive, performing small file writes. We find that Filebench introduces a severe scalability bottleneck (by locking the whole fileset to choose a file for operations like `open`) for Webproxy and Varmail. We bypass it by assigning a private fileset to each thread. However, we cannot increase the fileset count beyond sixteen due to a bug in Filebench. Due to time limitations, we evaluate Webproxy and Varmail with only up to sixteen threads.

Figure 11 shows the result. For Fileserver and Webserver, with one NUMA node, all the file systems perform similarly. With eight NUMA nodes, ARCKFS outperforms others by 1.1 $\times$ to 27.3 $\times$, and 1.2 $\times$ to 4.1 $\times$, respectively. These results are consistent with results in §6.3. Again, ARCKFS and OdinFS outperform thanks to their datapath design that maximizes NVM performance, and ARCKFS outperforms OdinFS due to direct NVM access and efficient metadata path (especially in Webserver).

Due to direct NVM access and efficient metadata operations, ARCKFS significantly outperforms others, including OdinFS, with workloads performing many metadata operations and small file accesses. Specifically, for Webproxy and Varmail, ARCKFS outperforms others by 2.2 $\times$ to 8.0 $\times$ and 2.4 $\times$ to 34.2 $\times$, respectively.

LevelDB. We evaluate LevelDB by running `db_bench` with the default setup; `db_bench` runs with one thread, the value size is 100 bytes, and there are one million objects in the database. As shown in Table 7, ARCKFS outperforms all the evaluated file systems across all the workloads. ARCKFS outperforms the second-best performant file system: WineFS, by up to 3.1 $\times$ and ext4 by 1.5 $\times$ to 17 $\times$. With `Fill100K`, ARCKFS-nd is 29% slower than ARCKFS. This is because `Fill100K` performs large file writes, and ARCKFS benefits from the access parallelization enabled by the opportunistic delegation (§4.6). On the other hand, other workloads mostly perform small file accesses,

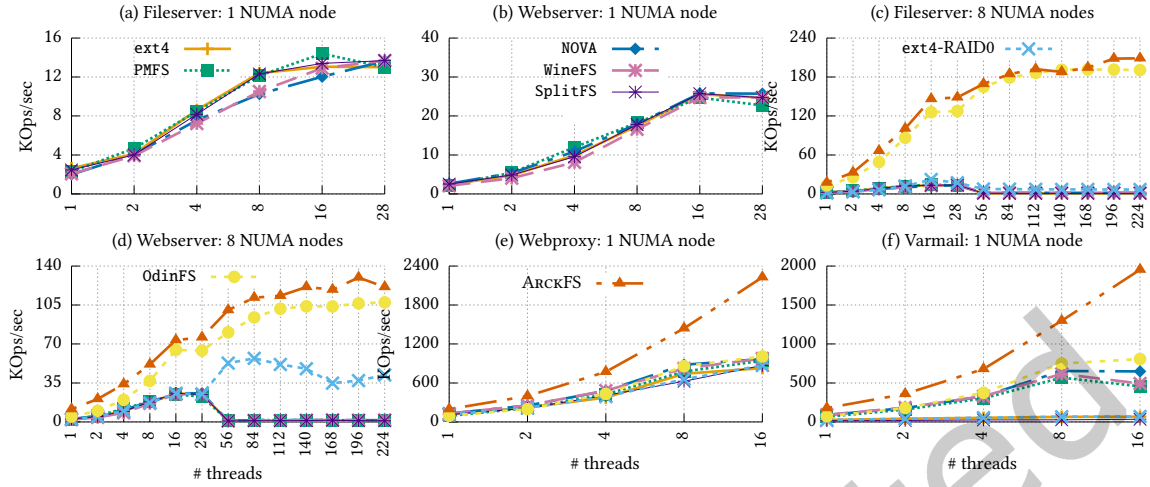


Fig. 11. Filebench results. ARCKFS consistently outperforms other evaluated file systems in various workloads. The one NUMA node results of Webproxy and Varmail are similar to those with eight NUMA nodes and thus omitted.

Throughput (ops/ms)	ext4	NOVA	WineFS	ARCKFS	ARCKFS-nd
Fill 100K	1.23	2.53	2.60	3.81	2.71
Fill seq	135	210	239	419	561
Fill sync	17	189	211	291	378
Fill random	123	196	219	343	452
Read random	93	131	142	144	174
Delete random	148	217	245	494	603

Table 7. Performance of LevelDB with the evaluated file systems. ext4(RAID0) always underperforms ext4, and is thus omitted. We cannot evaluate other file systems since they do not implement (functional) `mmap()`.

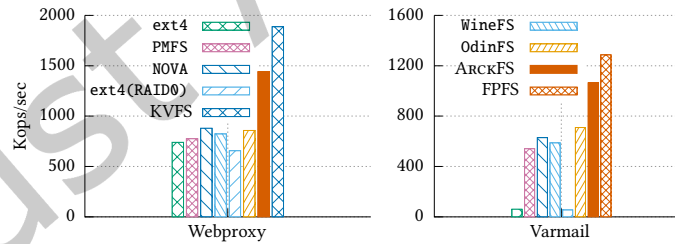


Fig. 12. Customized file systems enabled by TRIO: KVFS and FPFS further outperform ARCKFS.

where the opportunistic delegation incurs the striping and communication overhead (§6.2). As a result, ARCKFS-nd outperforms ARCKFS by 21% to 34%.

Customization. We design two customized file systems: KVFS and FPFS (§5) using the TRIO architecture. Figure 12 demonstrates the performance benefits of customization. We extend Filebench with a key-value interface to support KVFS. We create a directory depth of 20 in Varmail to stress path resolution. Both workloads run with eight threads. In Webproxy, KVFS avoids managing file descriptors and file indexes, further outperforming

- D. Zhou et al.

ARCKFS by 1.3× and others by 2.9×. In Varmail, FPFS supports full path indexing and thus further outperforms ARCKFS by 1.2× and others by 21×.

Summary. ARCKFS consistently outperforms other NVM file systems in macrobenchmarks that cover a wide range of file system use cases. The flexible customization enabled by TRIO can further improve application performance.

7 Other related work

TRIO follows Exokernel [40, 46] to provide applications with secure direct access to storage devices and a minimal abstraction for customization. Unlike TRIO, Exokernel assumes a conventional disk and thus still requires kernel mediation for disk accesses. Furthermore, Exokernel cannot enable sharing among customized LibFSes and cannot enforce the metadata integrity of a POSIX-like file system. Arrakis [62] also assumes a conventional disk, leverages hardware extension to partition the disk statically, and assigns applications a portion of the storage device for direct access. Thus, accessing a file in another application’s region requires inter-process communication. TRIO resolves the four way tension among direct access, customization, sharing, and security on the emerging NVM storage device.

State separation. ScaleFS [31] decouples an in-memory file system from an on-disk file system using operation logs. Unlike ScaleFS, TRIO only has a single layer of file systems: LibFSes and LibFSes directly accesses NVM, thereby avoiding the disadvantages of a log-based design (§2.3.1). NOVA [71] and Flatstore [32] maintain logs on NVM and maintain indexes in DRAM to improve performance. TRIO generalizes this design philosophy, applies it to userspace NVM file systems, and solves different problems.

Verifying file system consistency. SQCK [44] is a file system checker based on SQL to achieve simplicity and better functionality than e2fsck. TRIO’s integrity verifier performs similarly to Recon [42], which pioneers enforcing metadata integrity online and locally.

8 Conclusion

This paper presents TRIO, a new userspace NVM file system architecture that unleashes its performance potential and ensures metadata integrity. By separating the file system state into a single commonly shared core state and private auxiliary state in each component, TRIO simultaneously allows LibFSes to directly access NVM, applications to flexibly customize its LibFSes without special privileges or affecting other applications, and untrusted applications can securely share a file with its private LibFSes. Leveraging TRIO, we design ARCKFS, a generic POSIX-like file system with a careful design to achieve low latency, high throughput, and excellent scalability for both data and metadata operations. Our extensive evaluation shows that ARCKFS constantly outperforms existing NVM file systems by several times to orders of magnitude, while the customized file systems enabled by TRIO further outperform ARCKFS.

References

- [1] [n. d.]. 3D XPoint: A Breakthrough in Non-Volatile Memory Technology. <https://www.intel.com/content/www/us/en/architecture-and-technology/intel-micron-3d-xpoint-webcast.html>.
- [2] [n. d.]. Arm Memory System Resource Partitioning and Monitoring (MPAM) System Component Specification. <https://developer.arm.com/documentation/ih0099/latest/>.
- [3] [n. d.]. Compute Express Link 2.0 White Paper. https://b373eaf2-67af-4a29-b28c-3aae9e644f30.filesusr.com/ugd/0c1418_4c5283e7f3e40f9b2955c7d0f60bebe.pdf.
- [4] [n. d.]. Compute Express Link: The Breakthrough CPU-to-Device Interconnect. <https://www.computeexpresslink.org/download-the-specification>.
- [5] [n. d.]. dm-stripe. <https://www.kernel.org/doc/html/latest/admin-guide/device-mapper/striped.html>.

- [6] [n. d.]. Exim Internet Mailer. <https://www.exim.org/>.
- [7] [n. d.]. ext4(5) — Linux manual page. <https://man7.org/linux/man-pages/man5/ext4.5.html>.
- [8] [n. d.]. Filebench - A Model Based File System Workload Generator . <https://github.com/filebench/filebench>.
- [9] [n. d.]. Flexible I/O Tester. <https://github.com/axboe/fio>.
- [10] [n. d.]. fsck.ext4(8) - Linux man page. <https://linux.die.net/man/8/fsck.ext4>.
- [11] [n. d.]. HPE Persistent Memory. <https://www.hpe.com/us/en/servers/persistent-memory.html>.
- [12] [n. d.]. Intel Optane Persistent Memory. <https://www.intel.com/content/www/us/en/products/docs/memory-storage/optane-persistent-memory/overview.html>.
- [13] [n. d.]. Introduction to Memory Bandwidth Allocation. <https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-memory-bandwidth-allocation.html>.
- [14] [n. d.]. Last week Intel killed Optane. Today, Kioxia and Everspin announced comparable tech: Rumors of storage-class memory's demise may have been premature. https://www.theregister.com/2022/08/02/kioxia_everspin_persistent_memory/.
- [15] [n. d.]. Memory Protection Keys. <https://www.kernel.org/doc/html/latest/core-api/protection-keys.html>.
- [16] [n. d.]. Quota subsystem. <https://www.kernel.org/doc/html/latest/filesystems/quota.html>.
- [17] [n. d.]. Samsung Memory-Semantic SSD. <https://news.samsung.com/global/samsung-electronics-unveils-far-reaching-next-generation-memory-solutions-at-flash-memory-summit-2022>.
- [18] [n. d.]. Small Files, Big Foils: Addressing the Associated Metadata and Application Challenges. <https://blog.cloudera.com/small-files-big-foils/>.
- [19] [n. d.]. The Challenge in Big Data is Small Files. <https://blog.min.io/challenge-big-data-small-files/>.
- [20] [n. d.]. util-linux. <https://github.com/util-linux/util-linux>.
- [21] 2014. *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Broomfield, Colorado.
- [22] 2014. *Proceedings of the 9th European Conference on Computer Systems (EuroSys)*. Amsterdam, The Netherlands.
- [23] 2015. *Proceedings of the 25th ACM Symposium on Operating Systems Principles (SOSP)*. Monterey, CA.
- [24] 2017. *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*. Shanghai, China.
- [25] 2019. *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*. Ontario, Canada.
- [26] 2021. *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*. Virtual.
- [27] 2022. *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Carlsbad, CA.
- [28] 2022. *Proceedings of the 20th USENIX Conference on File and Storage Technologies (FAST)*. Santa Clara, CA.
- [29] 2023. *Proceedings of the 21th USENIX Conference on File and Storage Technologies (FAST)*. Santa Clara, CA.
- [30] Adam Belay, George Prekas, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. 2014. IX: A Protected Dataplane Operating System for High Throughput and Low Latency, See [21].
- [31] Srivatsa S. Bhat, Rasha Egbal, Austin T. Clements, M. Frans Kaashoek, and Nickolai Zeldovich. 2017. Scaling a file system to many cores using an operation log, See [24].
- [32] Youmin Chen, Youyou Lu, Fan Yang, Qing Wang, Yang Wang, and Jiwu Shu. 2020. FlatStore: An Efficient Log-Structured Key-Value Storage Engine for Persistent Memory. In *Proceedings of the 25th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Lausanne, Switzerland.
- [33] Youmin Chen, Youyou Lu, Bohong Zhu, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Jiwu Shu. 2021. Scalable Persistent Memory File System with Kernel-Userspace Collaboration. In *Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST)*. Virtual.
- [34] Austin T Clements, M. Frans Kaashoek, and Nickolai Zeldovich. 2013. RadixVM: Scalable Address Spaces for Multithreaded Applications. In *Proceedings of the 8th European Conference on Computer Systems (EuroSys)*. Prague, Czech Republic.
- [35] Austin T. Clements, M. Frans Kaashoek, Nickolai Zeldovich, Robert T. Morris, and Eddie Kohler. 2013. The Scalable Commutativity Rule: Designing Scalable Software for Multicore Processors. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*. Farmington, PA.
- [36] Björn Daase, Lars Jonas Bollmeier, Lawrence Benson, and Tilmann Rabl. 2021. Maximizing Persistent Memory Bandwidth Utilization for OLAP Workloads. In *Proceedings of the 2021 ACM SIGMOD/PODS Conference*. Xi'an, Shaanxi, China.

- D. Zhou et al.

- [37] Dave Dice and Alex Kogan. 2019. BRAVO: Biased Locking for Reader-Writer Locks. In *Proceedings of the 2019 USENIX Annual Technical Conference (ATC)*. Renton, WA.
- [38] Mingkai Dong, Heng Bu, Jifei Yi, Benchao Dong, and Haibo Chen. 2019. Performance and Protection in the ZoFS User-space NVM File System, See [25].
- [39] Subramanya R Dullloor, Sanjay Kumar, Anil Keshavamurthy, Philip Lantz, Dheeraj Reddy, Rajesh Sankaran, and Jeff Jackson. 2014. System Software for Persistent Memory, See [22].
- [40] Dawson R. Engler, M. Frans Kaashoek, and James O'Toole J. 1995. Exokernel: an operating system architecture for application-level resource management. In *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP)*. Copper Mountain, CO.
- [41] Jose M. Faleiro and Daniel J. Abadi. 2015. Rethinking serializable multiversion concurrency control . In *Proceedings of the 41st International Conference on Very Large Data Bases (VLDB)*. Hawaii, USA.
- [42] Daniel Fryer, Kuei Sun, Rahat Mahmood, TingHao Cheng, Shaun Benjamin, Ashvin Goel, and Angela Demke Brown. 2012. Recon: Verifying File System Consistency at Runtime. In *Proceedings of the 10th USENIX Conference on File and Storage Technologies (FAST)*. San Jose, CA.
- [43] Bill Gervasi. 2022. A Persistent CXL Memory Module with DRAM Performance. In *Storage Developer Conference (SDC)*. SNIA. <https://storagedeveloper.org/conference/agenda/sessions/persistent-cxl-memory-module-dram-performance>.
- [44] Haryadi S. Gunawi, Abhishek Rajimwale, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2008. SQCK: A Declarative File System Checker. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. San Diego, CA.
- [45] Joseph Izraelevitz, Jian Yang, Lu Zhang, Juno Kim, Xiao Liu, Amirsaman Memaripour, Yun Joon Soh, Zixuan Wang, Yi Xu, Subramanya R. Dullloor, Jishen Zhao, and Steven Swanson. 2019. Basic Performance Measurements of the Intel Optane DC Persistent Memory Module. *arXiv preprint arXiv:1903.05714* (2019).
- [46] M. Frans Kaashoek, Dawson R. Engler, Gregory R. Ganger, Hector M. Briceno, Russell Hunt, David Mazieres, Thomas Pinckney, Robert Grimm, John Jannotti, and Kenneth Mackenzie. 1997. Exokernel: an operating system architecture for application-level resource management. In *Proceedings of the 16th ACM Symposium on Operating Systems Principles (SOSP)*. Saint-Malo, France.
- [47] Rohan Kadekodi, Saurabh Kadekodi, Soujanya Ponnappalli, Harshad Shirwadkar, Gregory R. Ganger, Aasheesh Kolli, and Vijay Chidambaram. 2021. WineFS: a hugepage-aware file system for persistent memory that ages gracefully, See [26].
- [48] Rohan Kadekodi, Se Kwon Lee, Sanidhya Kashyap, Taesoo Kim, Aasheesh Kolli, and Vijay Chidambaram. 2019. SplitFS: Reducing Software Overhead in File Systems for Persistent Memory, See [25].
- [49] Kostis Kaffes, Jack Tigar Humphries, David Mazières, and Christos Kozyrakis. 2021. Syrup: User-Defined Scheduling Across the Stack, See [26].
- [50] Rajat Kateja, Anirudh Badam, Sriram Govindan, Bikash Sharma, and Greg Ganger. 2018. Viyojit: Decoupling Battery and DRAM Capacities for Battery-Backed DRAM. In *Proceedings of the 44th ACM/IEEE International Symposium on Computer Architecture (ISCA)*. Toronto, Canada.
- [51] Jongseok Kim, Cassiano Campos, Joo-Young Hwang, Jinkyu Jeong, and Euseong Seo. 2021. Z-Journal: Scalable Per-Core Journaling. In *Proceedings of the 2021 USENIX Annual Technical Conference (ATC)*. Virtual.
- [52] Alex Kogan, Dave Dice, and Shady Issa. 2020. Scalable Range Locks for Scalable Address Spaces and Beyond. In *Proceedings of the 15th European Conference on Computer Systems (EuroSys)*. Crete, Greece.
- [53] Youngjin Kwon, Henrique Fingler, Tyler Hunt, Simon Peter, Emmett Witchel, and Thomas Anderson. 2017. Strata: A Cross Media File System, See [24].
- [54] Hayley LeBlanc, Shankara Pailoor, Om Saran K R E, Isil Dillig, James Bornholt, and Vijay Chidambaram. 2023. Chipmunk: Investigating Crash-Consistency in Persistent-Memory File Systems. In *Proceedings of the 18th European Conference on Computer Systems (EuroSys)*. Rome, Italy.
- [55] Changman Lee, Dongho Sim, Joo-Young Hwang, and Sangyeun Cho. 2015. F2FS: A New File System for Flash Storage. In *Proceedings of the 13th USENIX Conference on File and Storage Technologies (FAST)*. Santa Clara, CA.
- [56] Ruibin Li, Xiang Ren, Xu Zhao, Siwei He, Michael Stumm, and Ding Yuan. 2022. ctFS: Replacing File Indexing with Hardware Memory Translation through Contiguous File Allocation for Persistent Memory, See [28].
- [57] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. 2020. Aria: A Fast and Practical Deterministic OLTP Database. In *Proceedings of the 46th International Conference on Very Large Data Bases (VLDB)*. Tokyo, Japan.

- [58] Changwoo Min, Sanidhya Kashyap, Steffen Maass, Woonhak Kang, and Taesoo Kim. 2016. Understanding Manycore Scalability of File Systems. In *Proceedings of the 2016 USENIX Annual Technical Conference (ATC)*. Denver, CO.
- [59] Joontaek Oh, Seung Won Yoo, Hojin Nam, Changwoo Min, and Youjip Won. 2023. CJFS: Concurrent Journaling for Better Scalability, See [29].
- [60] Daejun Park and Dongkun Shin. 2017. iJournaling: Fine-Grained Journaling for Improving the Latency of Fsync System Call. In *Proceedings of the 2017 USENIX Annual Technical Conference (ATC)*. Santa Clara, CA.
- [61] Sujin Park, Diyu Zhou, Yuchen Qian, Irina Calciu, Taesoo Kim, and Sanidhya Kashyap. 2022. Application-Informed Kernel Synchronization Primitives, See [27].
- [62] Simon Peter, Jialin Li, Irene Zhang, Dan R. K. Ports, Doug Woos, Arvind Krishnamurthy, Thomas Anderson, and Timothy Roscoe. 2014. Arrakis: The Operating System is the Control Plane, See [21].
- [63] George Prekas, Marios Kogias, and Edouard Bugnion. 2017. ZygOS: Achieving Low Tail Latency for Microsecond-scale Networked Tasks, See [24].
- [64] Suchitra Raman and Steven McCanne. 1999. A Model, Analysis, and Protocol Framework for Soft State-based Communication. In *Proceedings of the 10th ACM SIGCOMM*. Cambridge, MA.
- [65] Yujie Ren, Changwoo Min, and Sudarsun Kannan. 2020. CrossFS: A Cross-layered Direct-Access File System. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Virtual, 18 pages.
- [66] Mendel Rosenblum and John K. Ousterhout. 1992. The Design and Implementation of a Log-Structured File System. *ACM Transactions on Computer Systems* (1992).
- [67] Chia-Che Tsai, Yang Zhan, Jayashree Reddy, Yizheng Jiao, Tao Zhang, and Donald E. Porter. 2015. How to Get More Value From Your File System Directory Cache, See [23].
- [68] Haris Volos, Sanketh Nalli, Sankarlingam Panneerselvam, Venkatanathan Varadarajan, Prashant Saxena, and Michael M. Swift. 2014. Aerie: Flexible File-System Interfaces to Storage-Class Memory, See [22].
- [69] Zixuan Wang, Xiao Liu, Jian Yang, Theodore Michailidis, Steven Swanson, and Jishen Zhao. 2020. Characterizing and Modeling Non-Volatile Memory Systems. In *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Virtual.
- [70] Jian Xu, Juno Kim, Amirsaman Memaripour, and Steven Swanson. 2019. Finding and Fixing Performance Pathologies in Persistent Memory Software Stacks. In *Proceedings of the 24th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Providence, RI, 15 pages.
- [71] Jian Xu and Steven Swanson. 2016. NOVA: A Log-structured File System for Hybrid Volatile/Non-volatile Main Memories. In *Proceedings of the 14th USENIX Conference on File and Storage Technologies (FAST)*. Santa Clara, CA.
- [72] Jian Xu, Lu Zhang, Amirsaman Memaripour, Akshatha Gangadharaiah, Amit Borase, Tamires Brito Da Silva, Steven Swanson, and Andy Rudof. 2017. NOVA-Fortis: A Fault-Tolerant Non-Volatile Main Memory File System, See [24].
- [73] Jian Yang, Juno Kim, Morteza Hoseinzadeh, Joseph Izraelevitz, and Steven Swanson. 2020. An Empirical Guide to the Behavior and Use of Scalable Persistent Memory. In *Proceedings of the 18th USENIX Conference on File and Storage Technologies (FAST)*. Santa Clara, CA.
- [74] Suli Yang, Tyler Harter, Nishant Agrawal, Salini Selvaraj Kowsalya, Anand Krishnamurthy, Samer Al-Kiswany, Rini T. Kaushik, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2015. Split-Level I/O Scheduling, See [23].
- [75] Jifei Yi, Benchao Dong, Mingkai Dong, Ruizhe Tong, and Haibo Chen. 2022. MT2: Memory Bandwidth Regulation on Hybrid NVM/DRAM Platforms, See [28].
- [76] Yang Zhan, Alex Conway, Yizheng Jiao, Eric Knorr, Michael A. Bender, Martin Farach-Colton, William Jannen, Rob Johnson, Donald E. Porter, and Jun Yuan. 2018. The Full Path to Full-Path Indexing. In *Proceedings of the 16th USENIX Conference on File and Storage Technologies (FAST)*. Oakland, CA.
- [77] Shawn Zhong, Chenhao Ye, Guanzhou Hu, Suyan Qu, Andrea Arpaci-Dusseau, Remzi Arpaci-Dusseau, and Michael Swift. 2023. MadFS: Per-File Virtualization for Userspace Persistent Memory Filesystems, See [29].
- [78] Diyu Zhou, Yuchen Qian, Vishal Gupta, Zhifei Yang, Changwoo Min, and Sanidhya Kashyap. 2022. Odinfs: Scaling PM performance with Opportunistic Delegation, See [27].

Received 30 May 2025; revised 20 February 2026; accepted 22 May 2026